



Machine Learning

Support Vector Machine (SVM)

Part 1: Motivation

Dr. Mehran Safayani


safayani@iut.ac.ir


https://people.iut.ac.ir/en/safayani


https://www.aparat.com/mehran.safayani

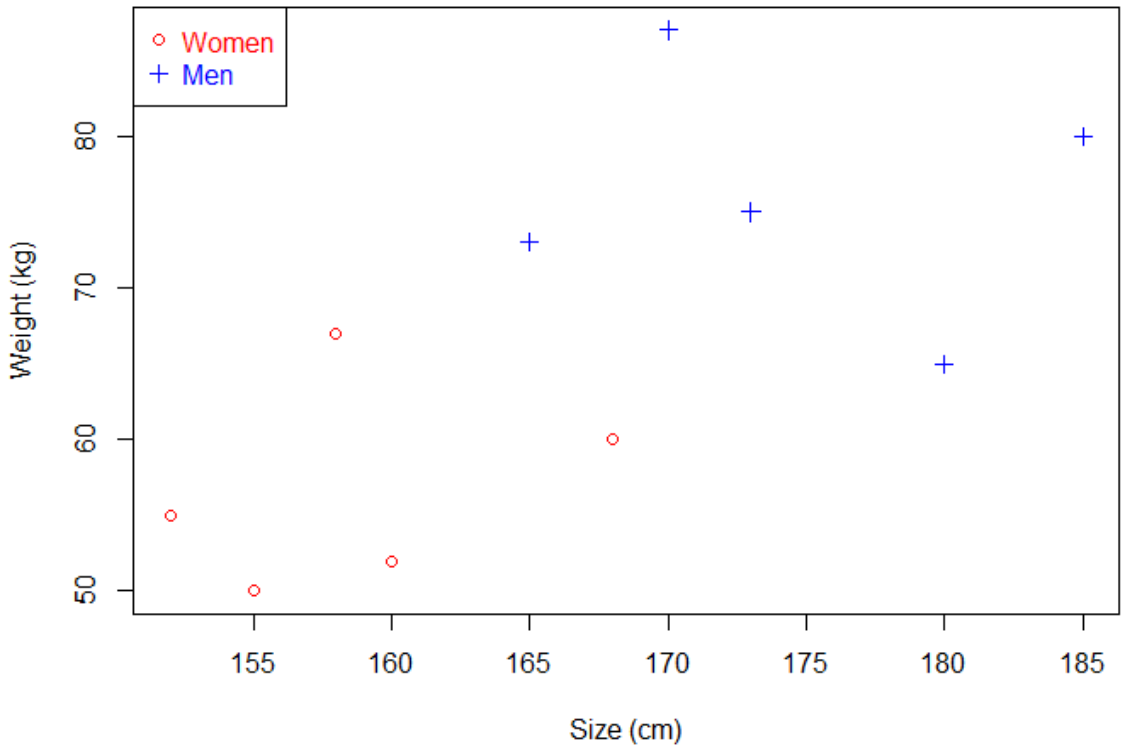

https://github.com/safayani/machine_learning_course

https://git.iut.ac.ir/safayani/courses/-/tree/main/Machine_Learning_course

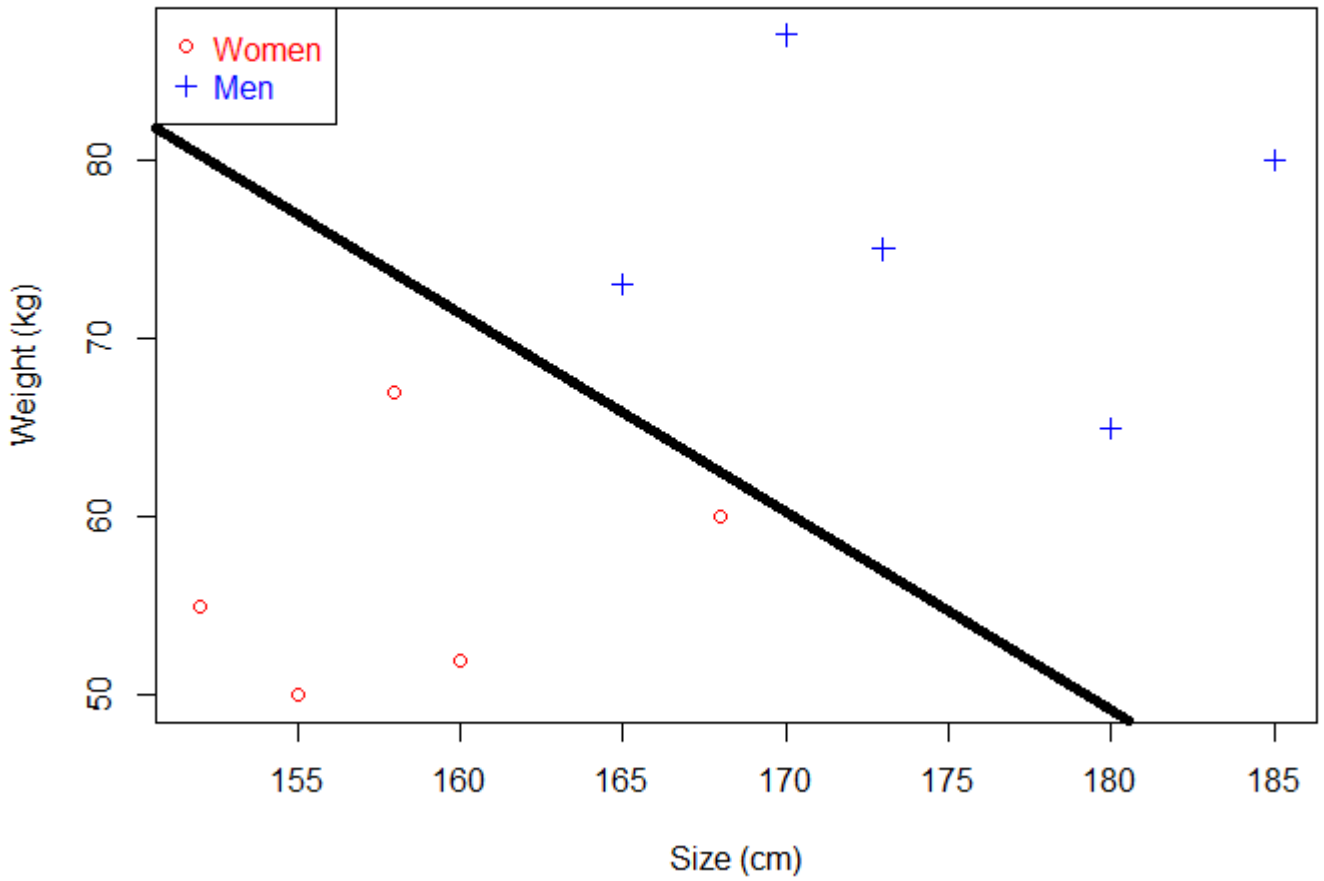


Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

SVM: Motivation



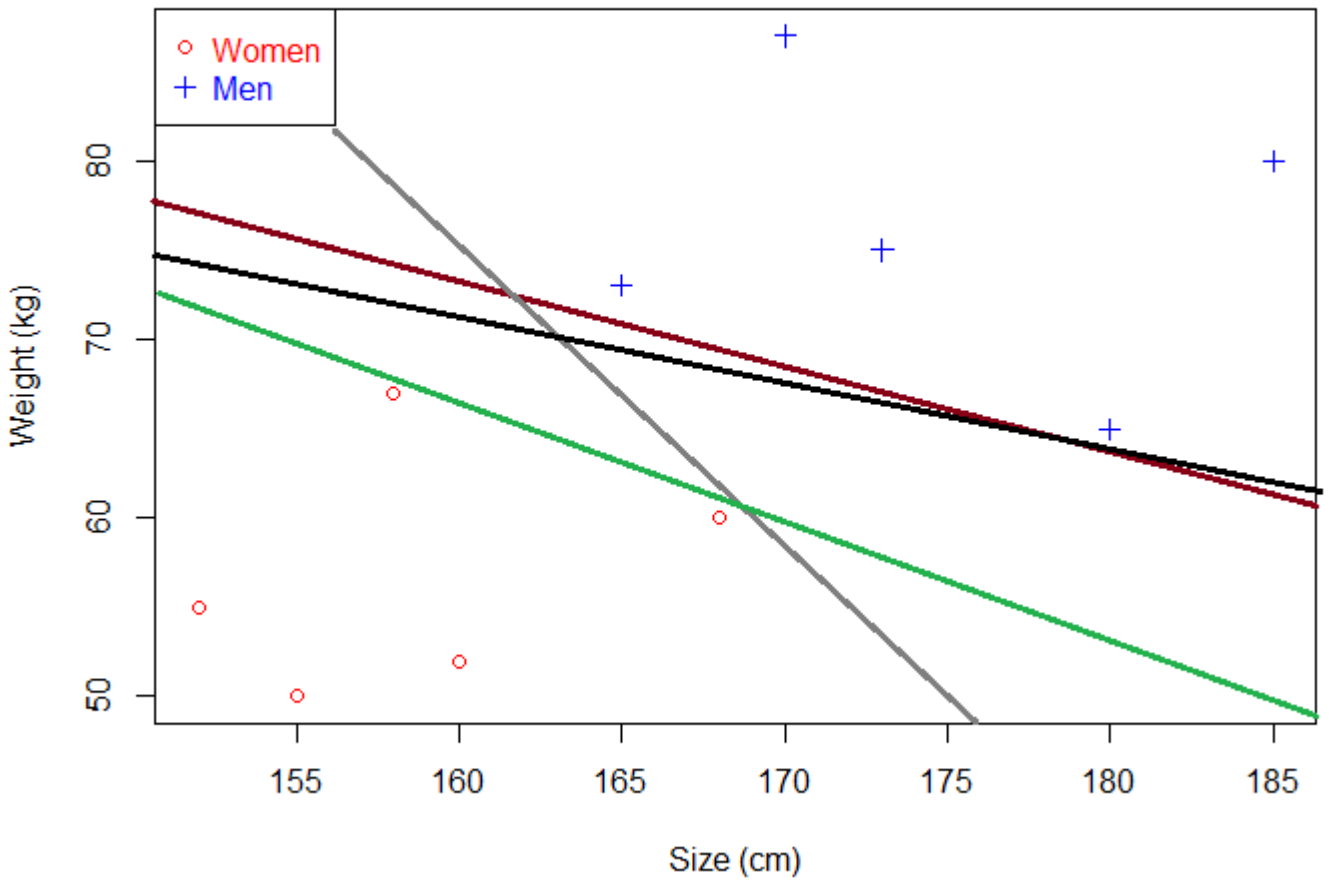
separating hyperplane



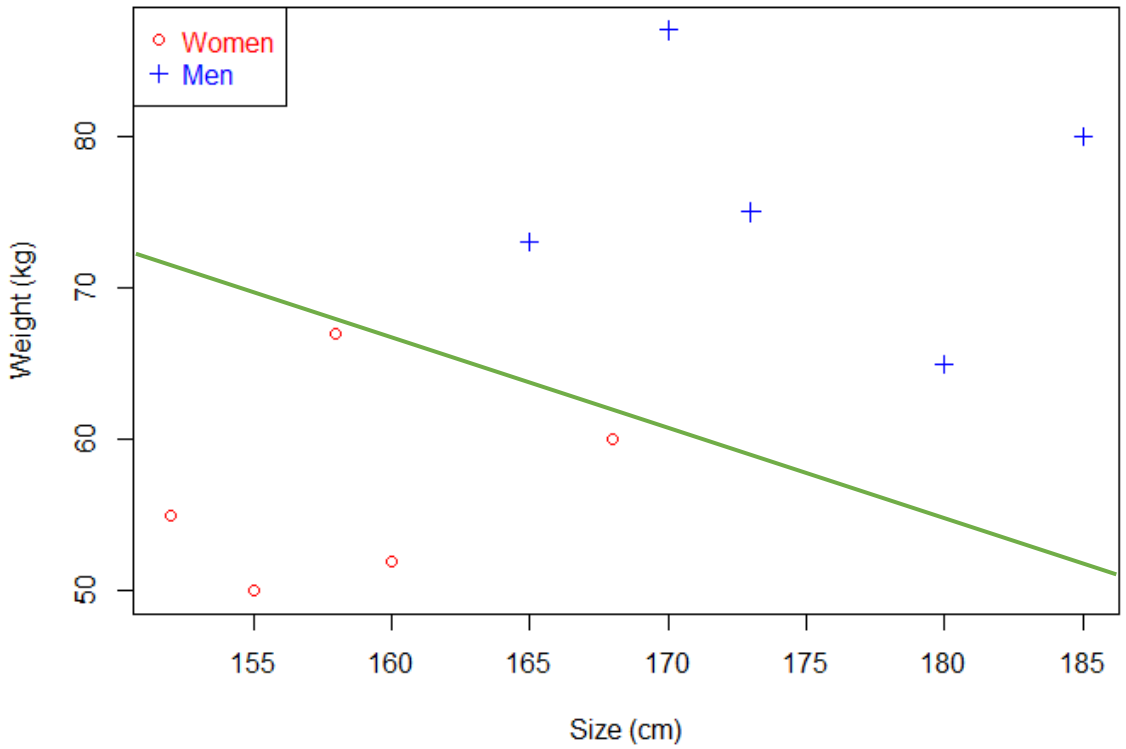




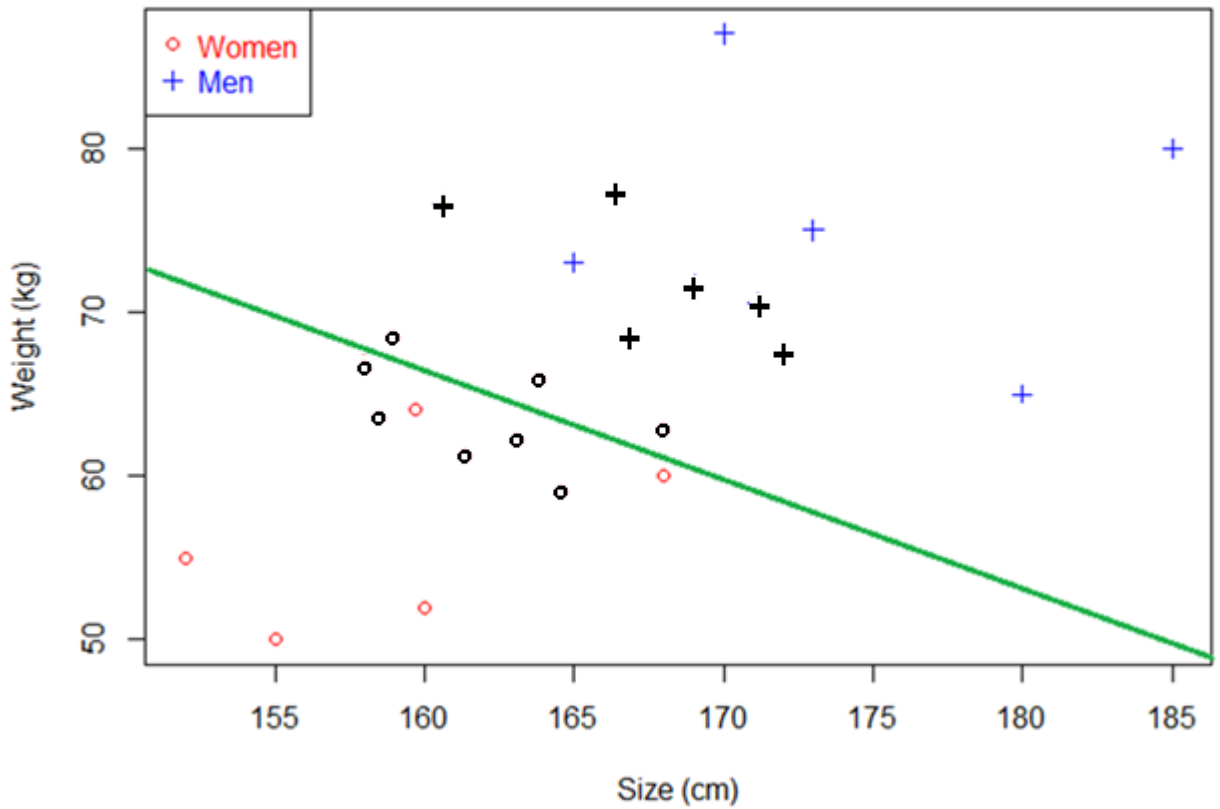
What is the *optimal* separating hyperplane?



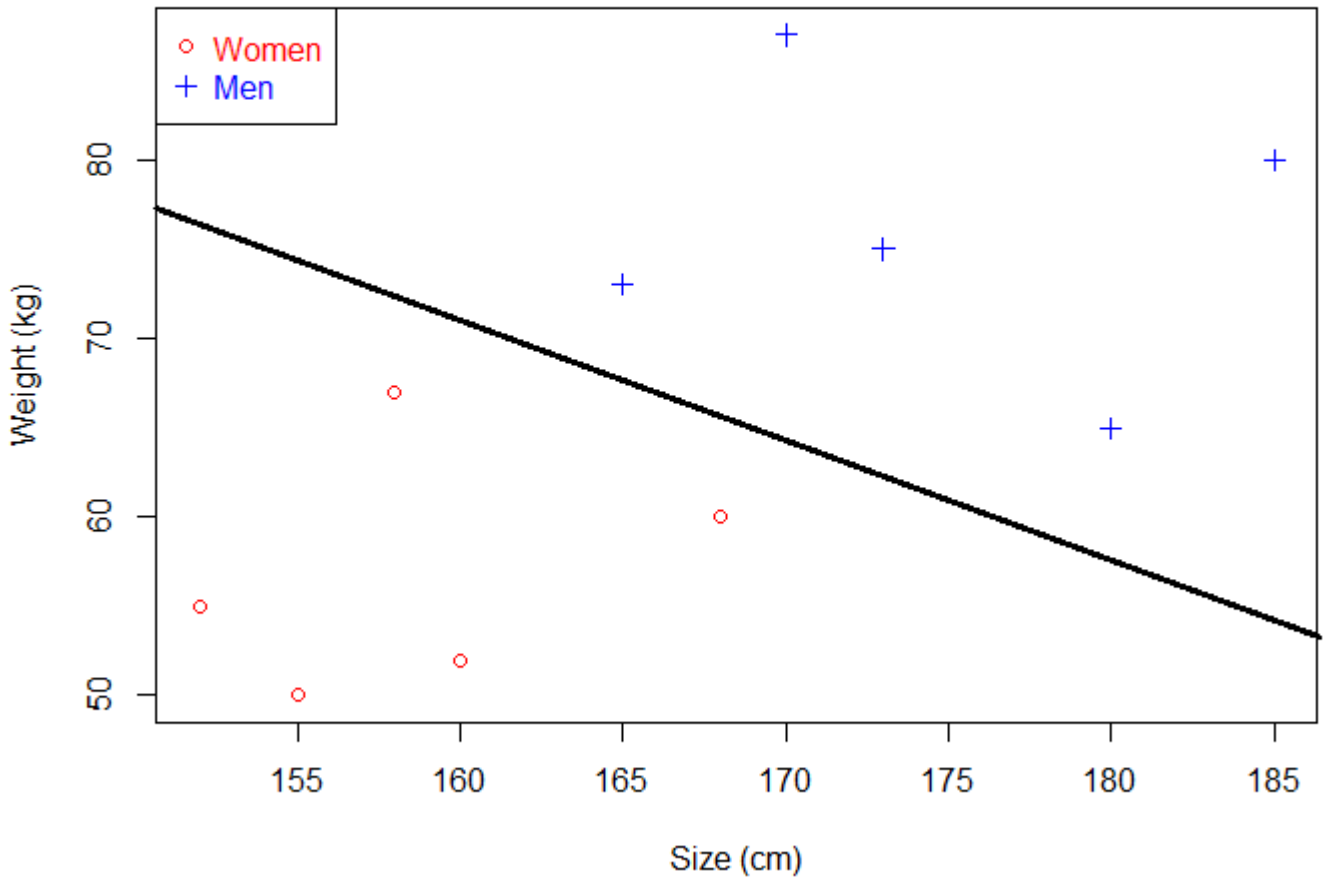
This hyperplane does not generalize well



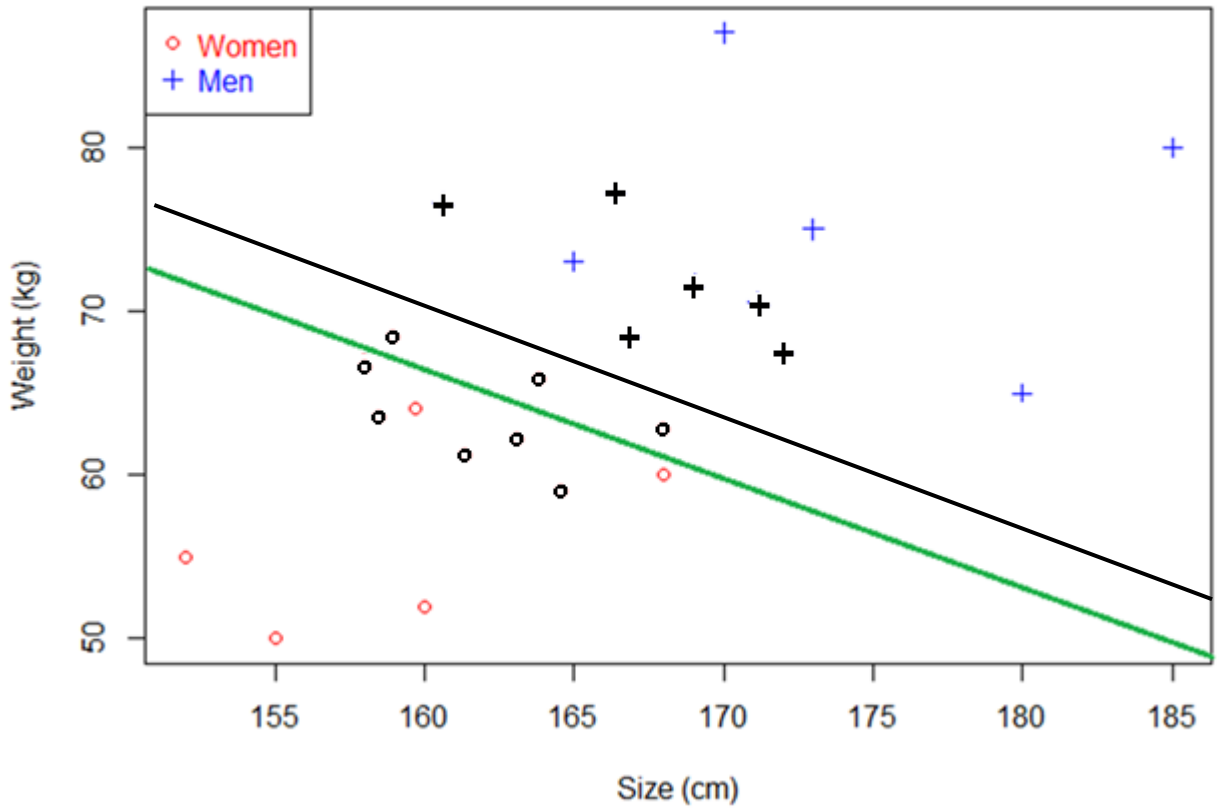
This hyperplane does not generalize well

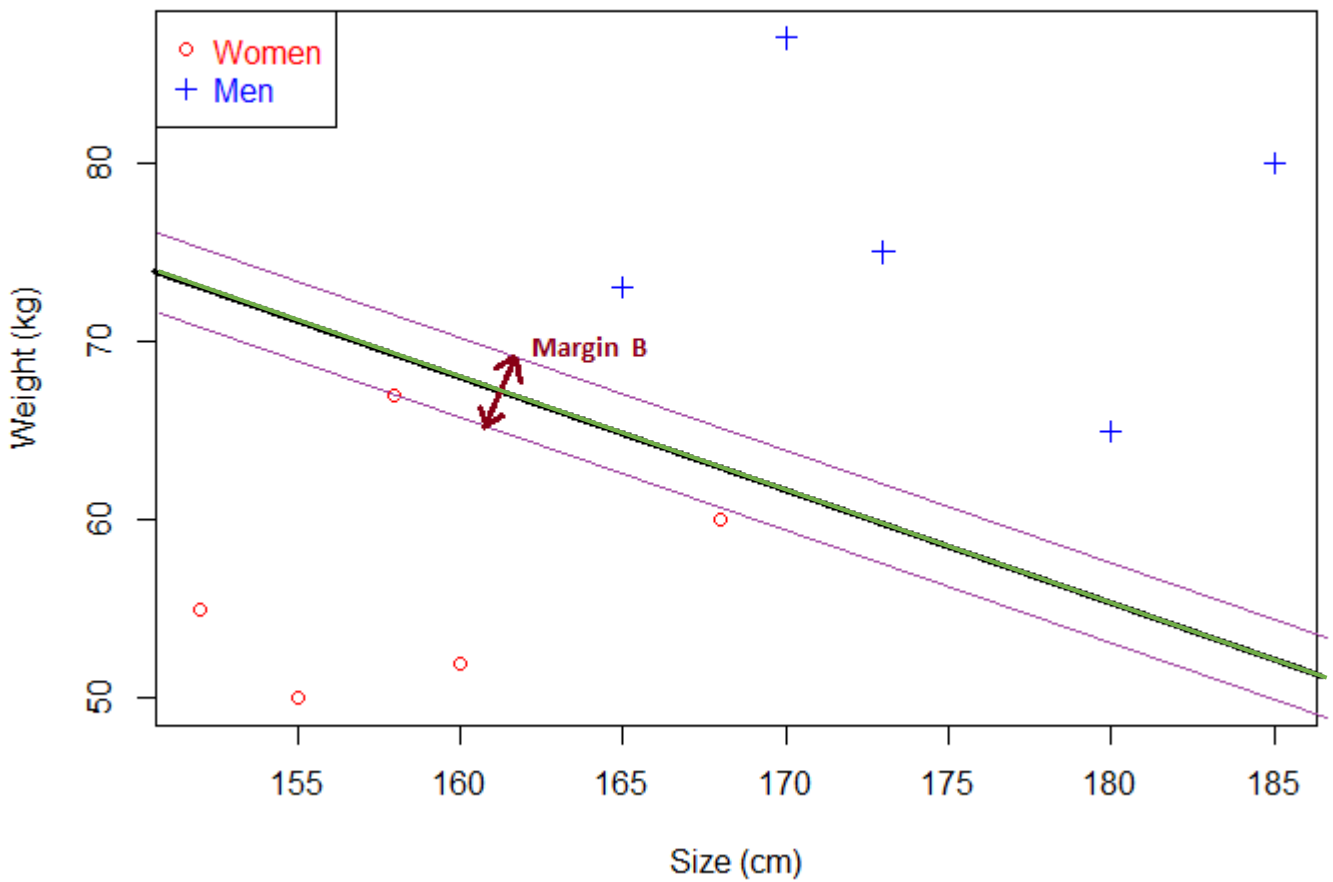


Optimal hyperplane

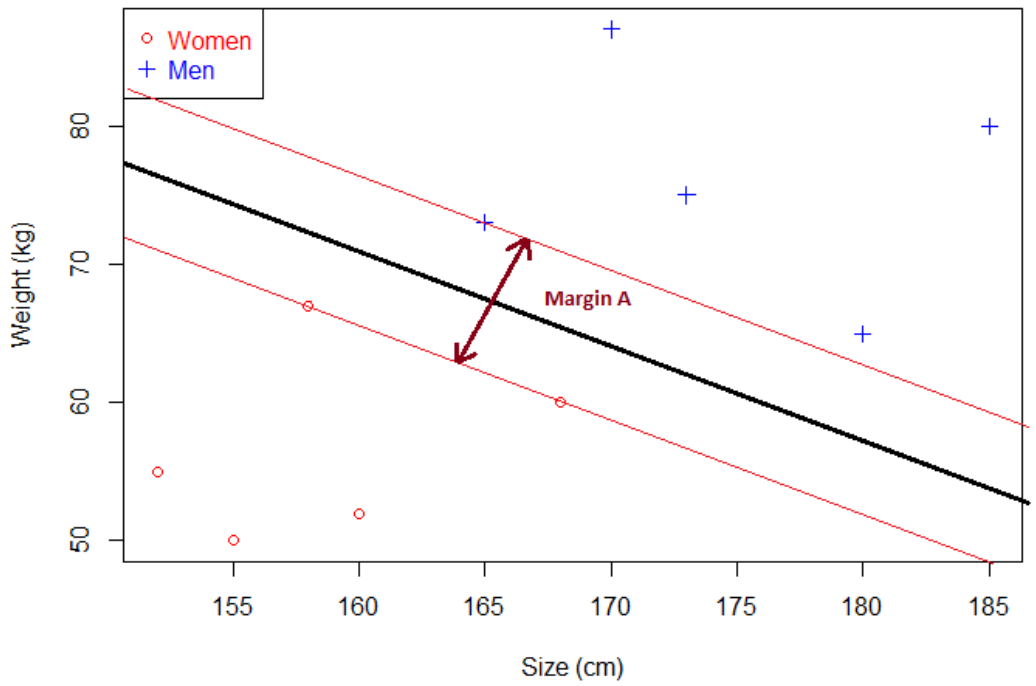


The black hyperplane classifies more accurately than the green one




Women
Men
Margin B
Weight (kg)
Size (cm)
50
60
70
80
155
160
165
170
175
180
185

The margin of our optimal hyperplane



- the optimal hyperplane will be the one with the biggest margin.

Machine Learning

Support Vector Machine (SVM)

Part 2: Reminder of linear algebra concepts

Dr. Mehran Safayani



safayani@iut.ac.ir

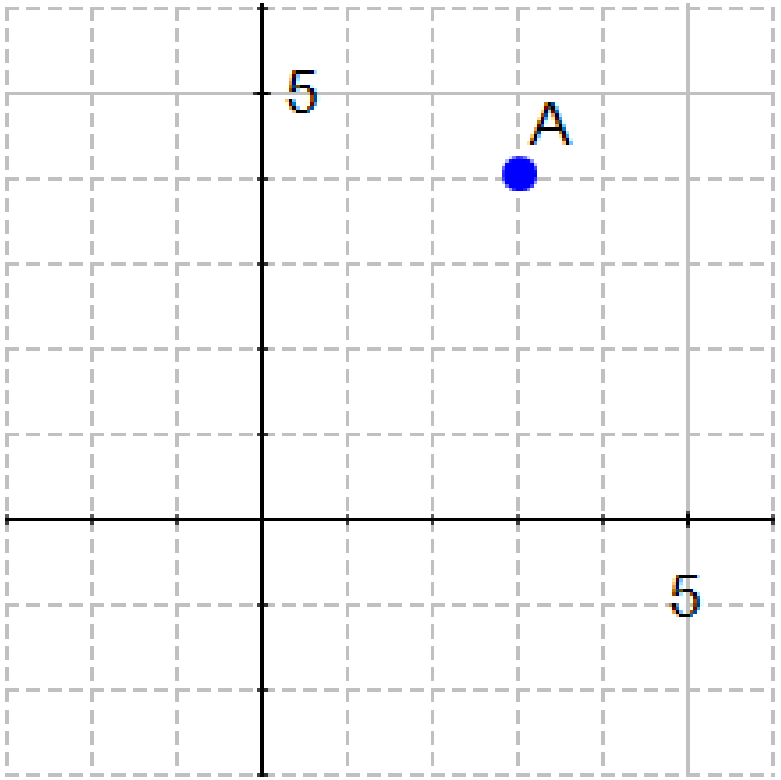
https://people.iut.ac.ir/en/safayani

https://www.aparat.com/mehran.safayani

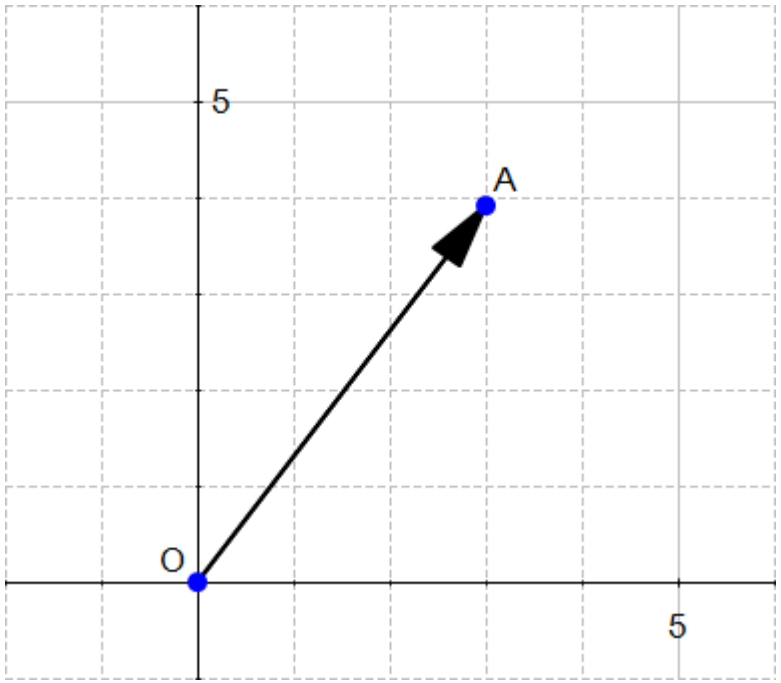
https://github.com/safayani/machine_learning_course

https://git.iut.ac.ir/safayani/courses/-/tree/main/Machine_Learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

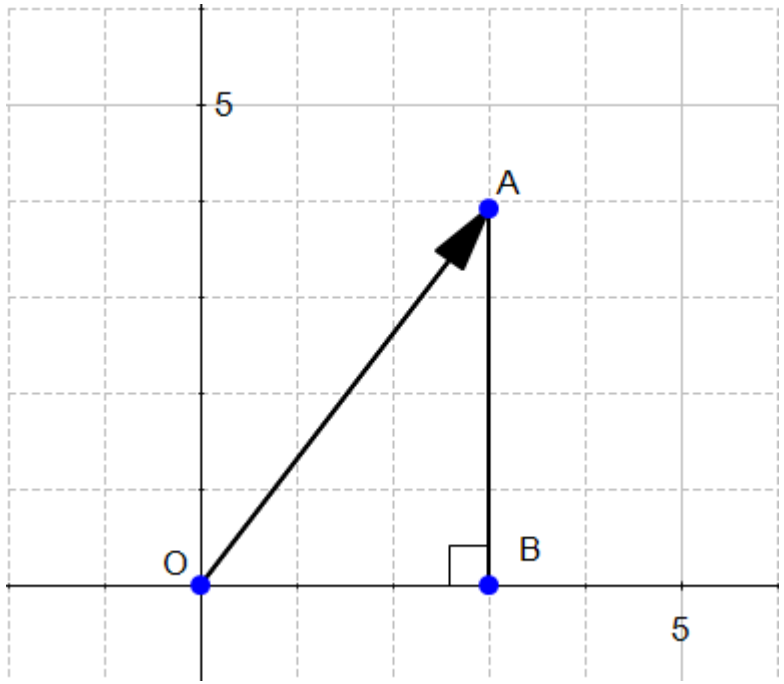


(3,4)



$\overrightarrow{OA}$

The magnitude



$$OA^2 = OB^2 + AB^2$$

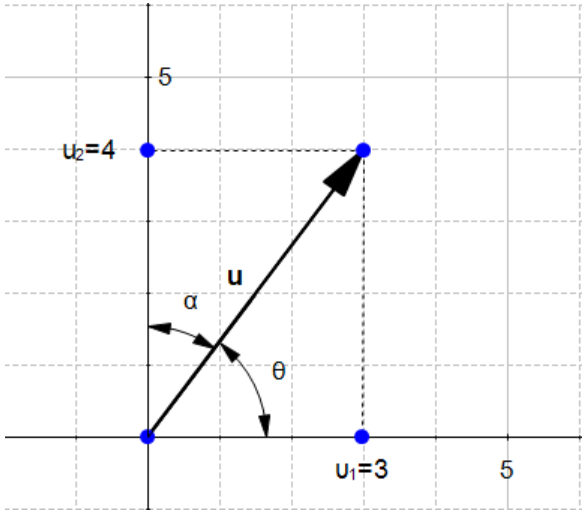
$$OA^2 = 3^2 + 4^2$$

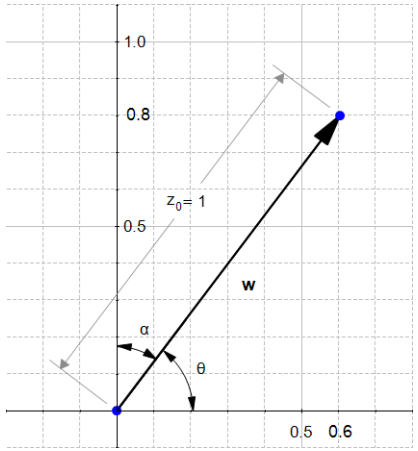
$$OA^2 = 25$$

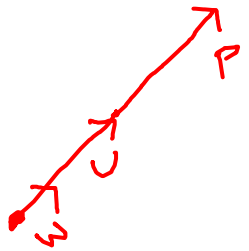
$$OA = \sqrt{25}$$

$$\|OA\| = OA = 5$$

The direction







$\vec{P} = t\vec{U} = (tu_1, tu_2)$

$\|P\| = t\|U\|$

$w = \left(\frac{u_1}{\|u\|}, \frac{u_2}{\|u\|}\right)$

$w^* = \left(\frac{P_1}{\|P\|}, \frac{P_2}{\|P\|}\right) = \left(\frac{tu_1}{t\|u\|}, \frac{tu_2}{t\|u\|}\right) = w$

*Definition : The **direction** of a vector* $\mathbf{u}(u_1, u_2)$ *is the vector* $\mathbf{w}\left(\frac{u_1}{\|u\|}, \frac{u_2}{\|u\|}\right)$

$$cos(\theta) = \frac{u_1}{\|u\|} = \frac{3}{5} = 0.6$$

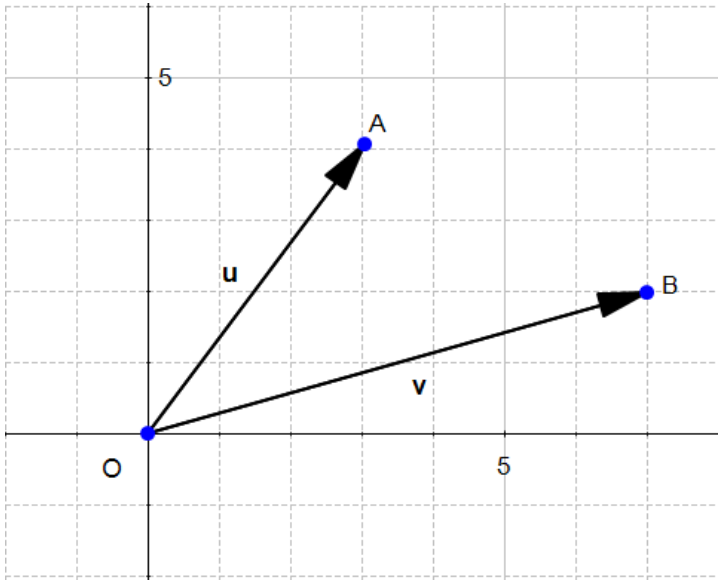
$$cos(\alpha) = \frac{u_2}{\|u\|} = \frac{4}{5} = 0.8$$

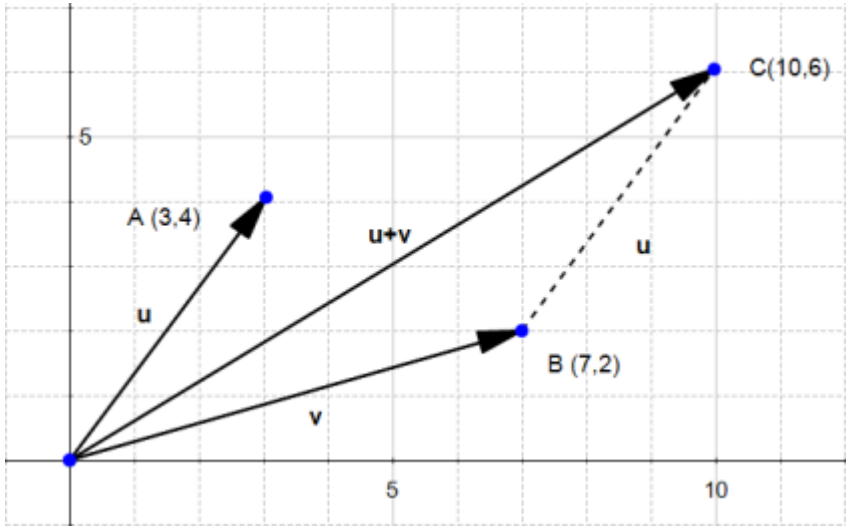
The direction of $\mathbf{u}(3, 4)$ is the vector $\mathbf{w}(0.6, 0.8)$

The sum of two vectors

Given two vectors $\mathbf{u}(u_1, u_2)$ and $\mathbf{v}(v_1, v_2)$ then :

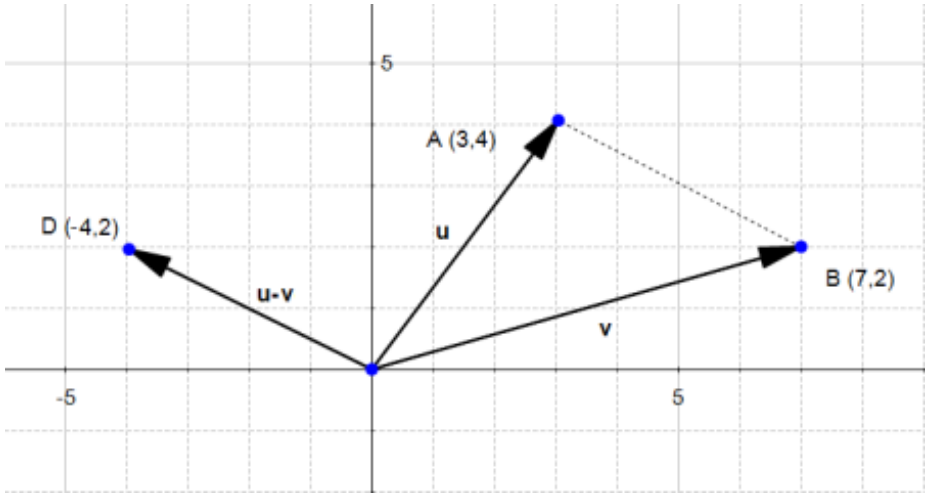
$$\mathbf{u} + \mathbf{v} = (u_1 + v_1, u_2 + v_2)$$

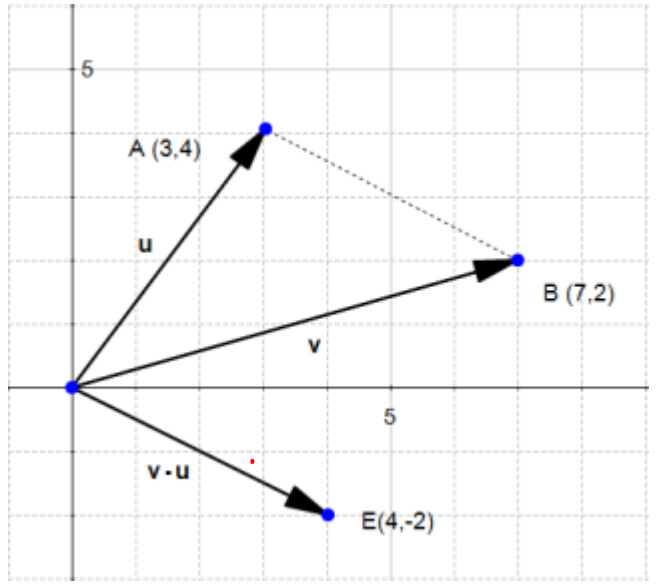




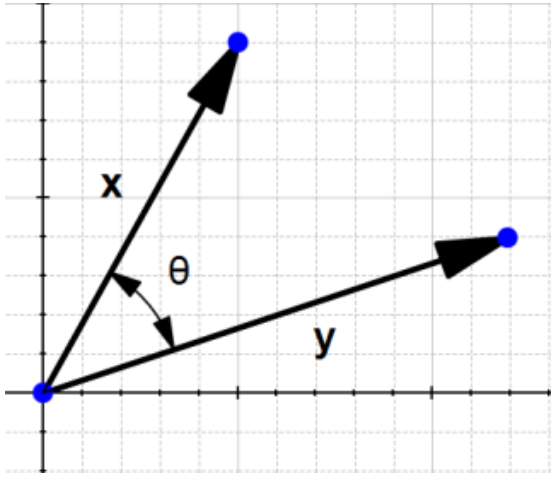
The difference between two vectors

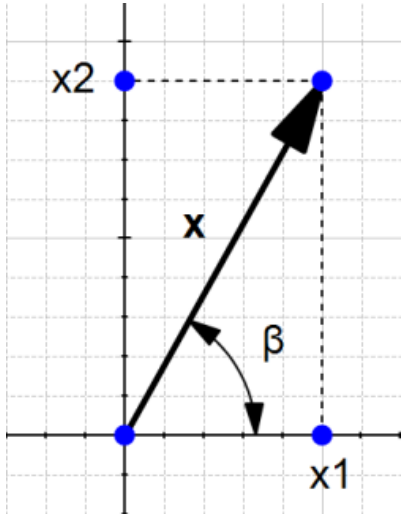
$$\mathbf{u} - \mathbf{v} = (u_1 - v_1, u_2 - v_2)$$

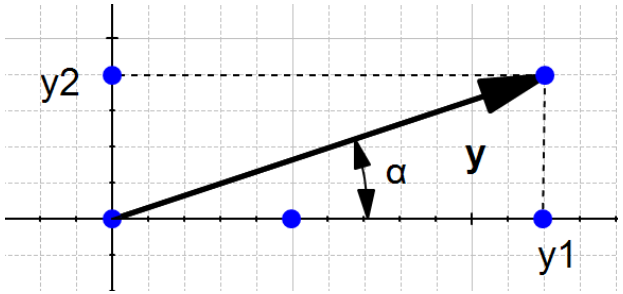




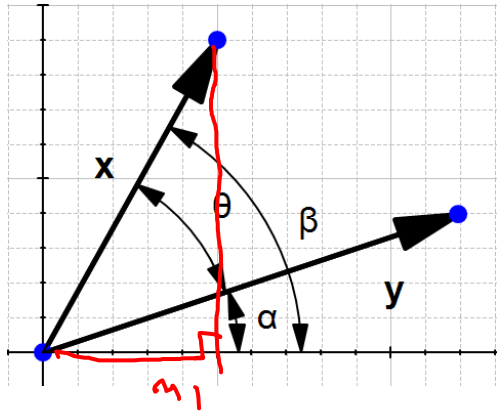
The dot product







Dot product



$$cos(\beta) = \frac{adjacent}{hypotenuse} = \frac{x_1}{\|x\|}$$

$$sin(\beta) = \frac{opposite}{hypotenuse} = \frac{x_2}{\|x\|}$$

$$cos(\alpha) = \frac{adjacent}{hypotenuse} = \frac{y_1}{\|y\|}$$

$$sin(\alpha) = \frac{opposite}{hypotenuse} = \frac{y_2}{\|y\|}$$

$$\theta = \beta - \alpha$$

$$cos(\theta) = cos(\beta - \alpha) = cos(\beta)cos(\alpha) + sin(\beta)sin(\alpha)$$

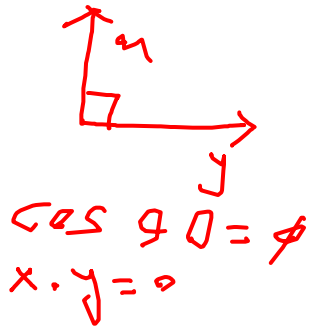
$$cos(\theta) = \frac{x_1}{\|x\|}\frac{y_1}{\|y\|} + \frac{x_2}{\|x\|}\frac{y_2}{\|y\|}$$

$$cos(\theta) = \frac{x_1y_1 + x_2y_2}{\|x\|\|y\|}$$

$$\|x\|\|y\|cos(\theta) = x_1y_1 + x_2y_2$$

$$\mathbf{x} \cdot \mathbf{y} = x_1y_1 + x_2y_2 = \sum_{i=1}^{2}(x_iy_i)$$

$$\|x\|\|y\|cos(\theta) = \mathbf{x} \cdot \mathbf{y}$$



Hyperplane

equation of a line is : $y = ax + b$

equation of an hyperplane is defined by : $\mathbf{w}^T\mathbf{x} = 0$

$$y - ax - b = 0$$

Given two vectors $\mathbf{w}\begin{pmatrix} -b \\ -a \\ 1 \end{pmatrix}$ and $\mathbf{x}\begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$

$$\mathbf{w}^T\mathbf{x} = -b \times (1) + (-a) \times x + 1 \times y$$

$$\mathbf{w}^T\mathbf{x} = y - ax - b$$





Hyperplane

Why do we use the hyperplane equation $\mathbf{w}^T\mathbf{x}$ instead of $y = ax + b$?

For two reasons:



- it is easier to work in more than two dimensions with this notation,
- the vector $\mathbf{w}$ will always be normal to the hyperplane

Machine Learning

Support Vector Machine (SVM)

Part 3: Linear SVM

Dr. Mehran Safayani



safayani@iut.ac.ir

https://people.iut.ac.ir/en/safayani

https://www.aparat.com/mehran.safayani

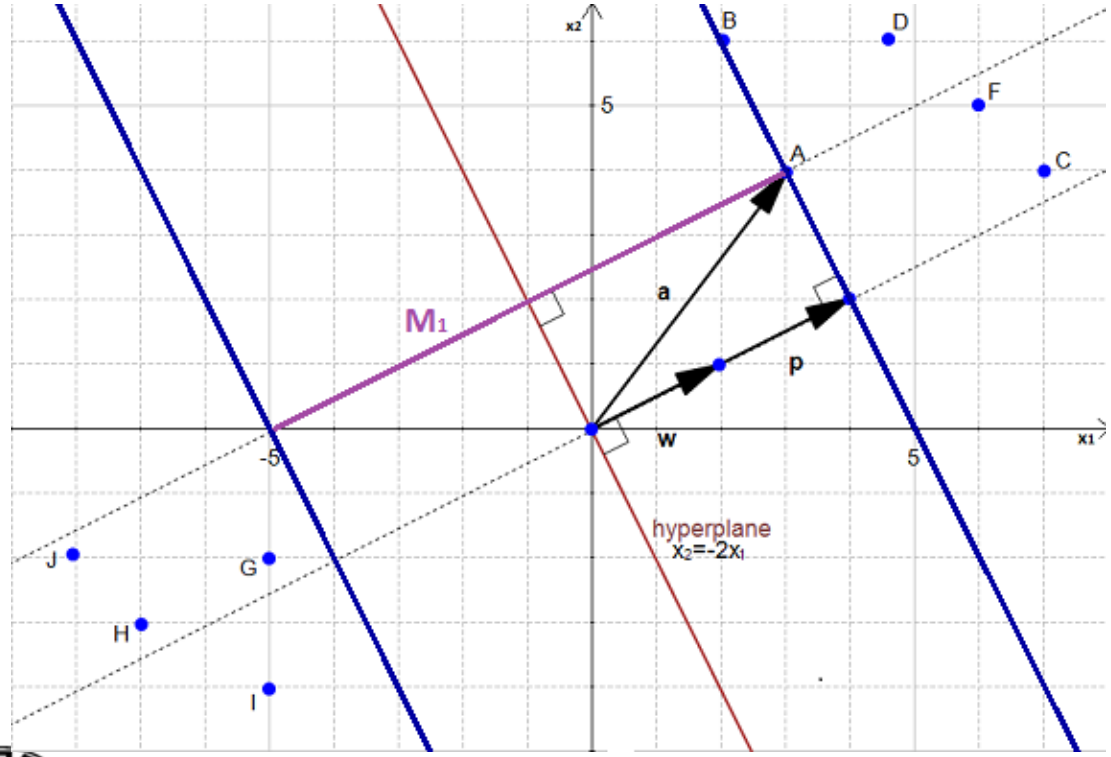
https://github.com/safayani/machine_learning_course

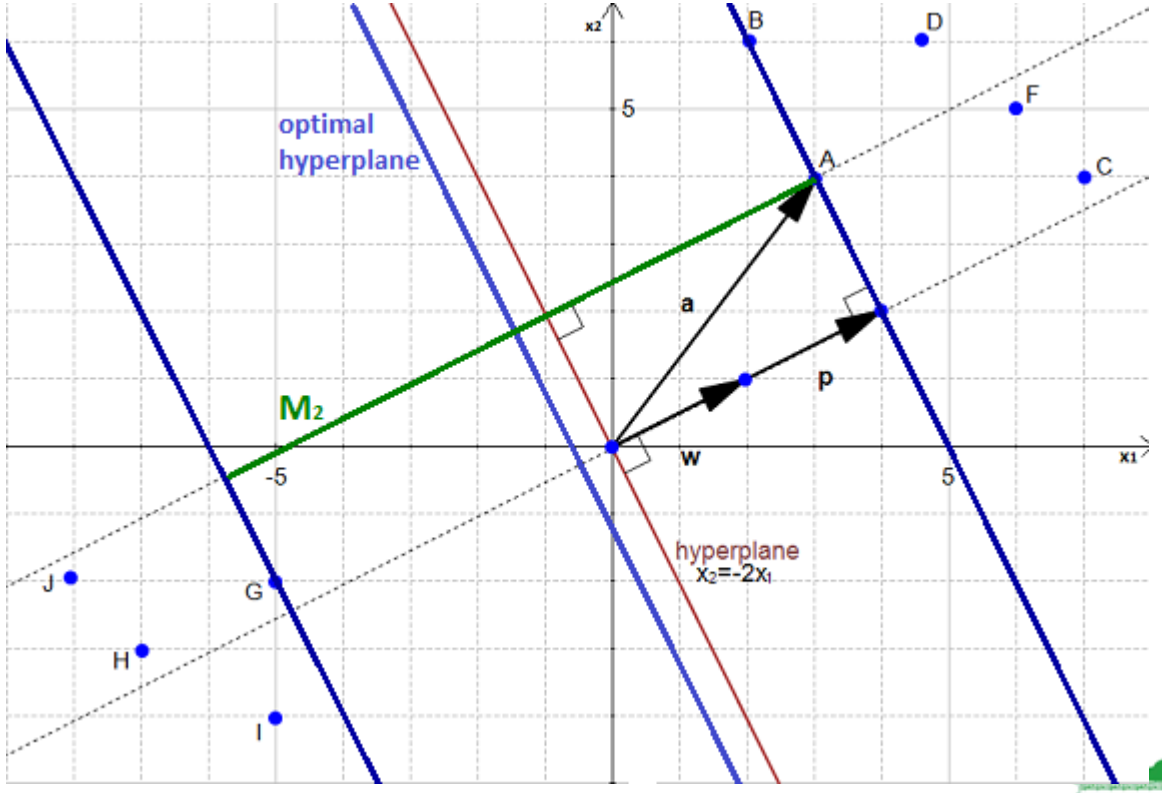
https://git.iut.ac.ir/safayani/courses/-/tree/main/Machine_Learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

the optimal hyperplane

- **How to find the optimal hyperplane ?**



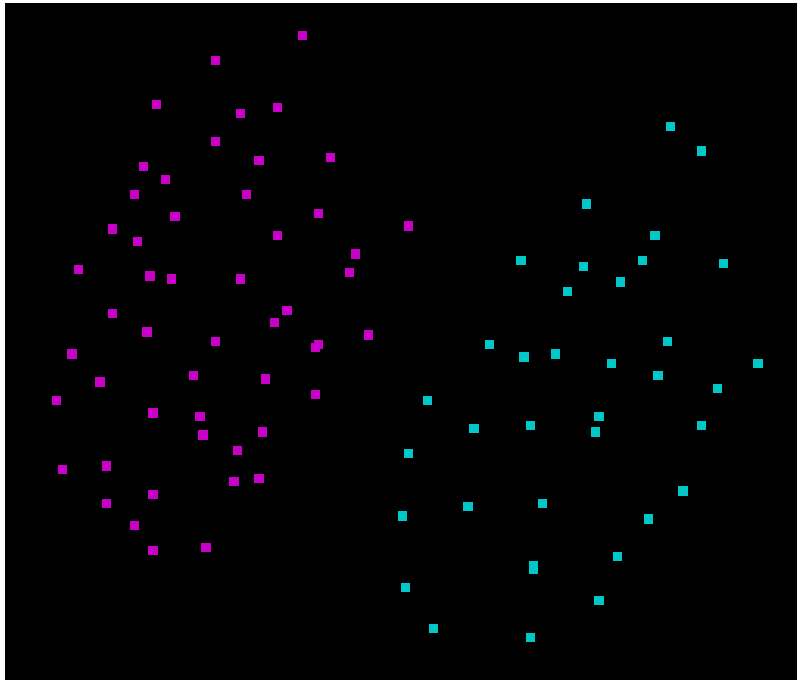


You have a dataset D and you want to classify it

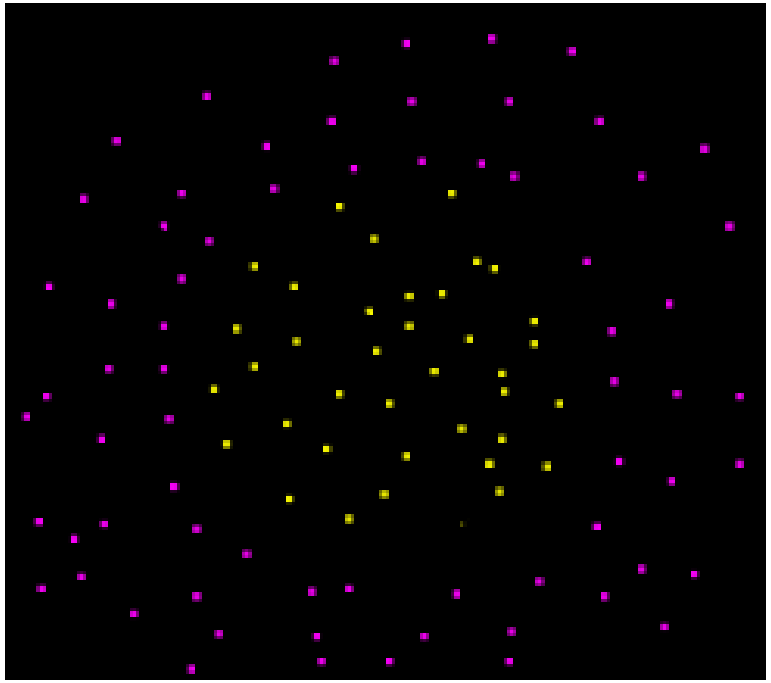
$$\mathcal{D} = \{(\mathbf{x}_i, y_i) \mid \mathbf{x}_i \in \mathbb{R}^p,\ y_i \in \{-1, 1\}\}_{i=1}^{n}$$







Linearly separable data



Non linearly separable data

let's assume that our dataset D IS linearly separable.





Taking another look at the hyperplane equation

$$\mathbf{w}^T\mathbf{x} = 0$$

$$\mathbf{w}(b, -a, 1) \text{ and } \mathbf{x}(1, x, y)$$

$$\mathbf{w} \cdot \mathbf{x} = b \times (1) + (-a) \times x + 1 \times y$$

$$\mathbf{w} \cdot \mathbf{x} = y - ax + b$$





$$\mathbf{w}'(-a, 1) \text{ and } \mathbf{x}'(x, y)$$

$$\mathbf{w}' \cdot \mathbf{x}' = (-a) \times x + 1 \times y$$

$$\mathbf{w}' \cdot \mathbf{x}' = y - ax$$

$$\mathbf{w}' \cdot \mathbf{x}' + b = y - ax + b$$

$$\mathbf{w}' \cdot \mathbf{x}' + b = \mathbf{w} \cdot \mathbf{x}$$

Given a hyperplane H_0 separating the dataset and satisfying:



$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

We can select two others hyperplanes H_1 and H_2 which also separate the data and have the following equations :

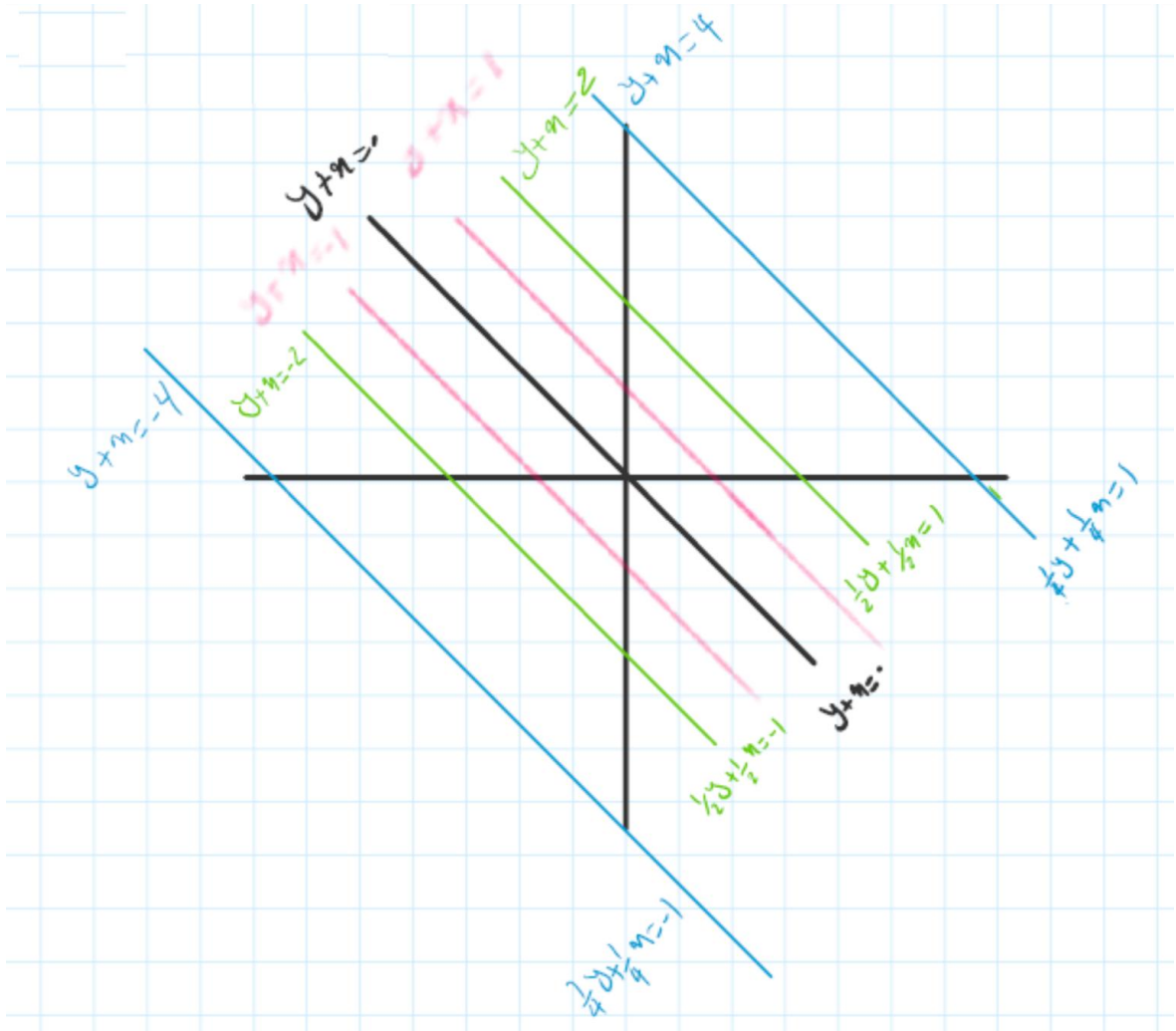
$$\mathbf{w} \cdot \mathbf{x} + b = \delta$$

$$\mathbf{w} \cdot \mathbf{x} + b = -\delta$$

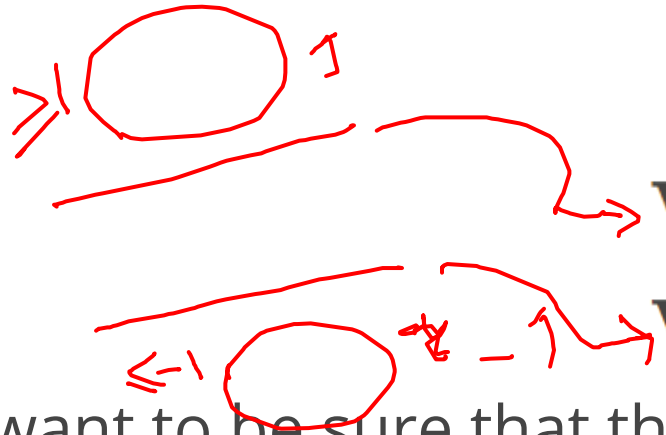
so that H_0 is equidistant from H_1 and H_2.






y+x=0
y+x=-1
y+x=2
y+x=-4
y+x=-2
y+x=4
y+x=1
y+x=-1
y+x=-3

However, here the variable δ is not necessary. So we can set $\delta = 1$ to simplify the problem.



$$\mathbf{w} \cdot \mathbf{x} + b = 1$$

$$\mathbf{w} \cdot \mathbf{x} + b = -1$$

Now we want to be sure that they have no points between them.

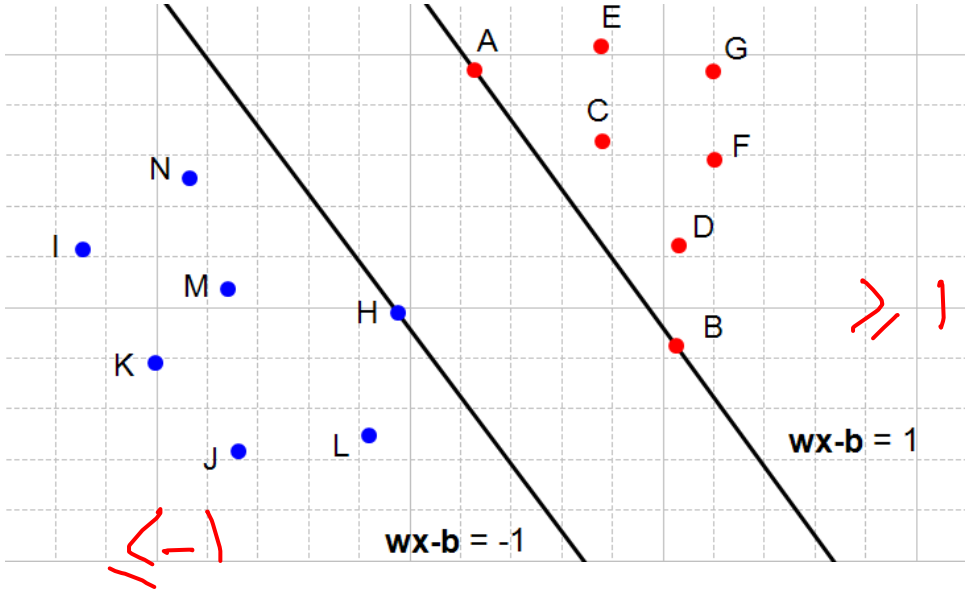
constraints:

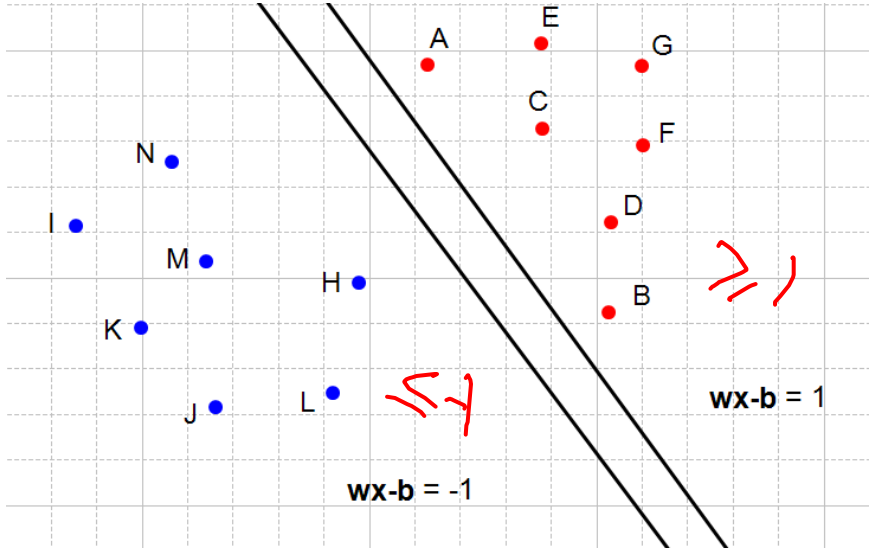
For each vector $\mathbf{x_i}$ either :

$$\mathbf{w} \cdot \mathbf{x_i} + b \geq 1 \text{ for } \mathbf{x_i} \text{ having the class } 1$$

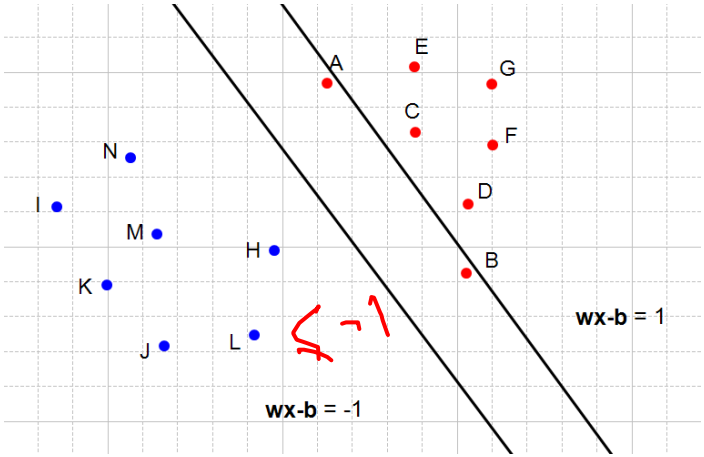
$$\mathbf{w} \cdot \mathbf{x_i} + b \leq -1 \text{ for } \mathbf{x_i} \text{ having the class } -1$$

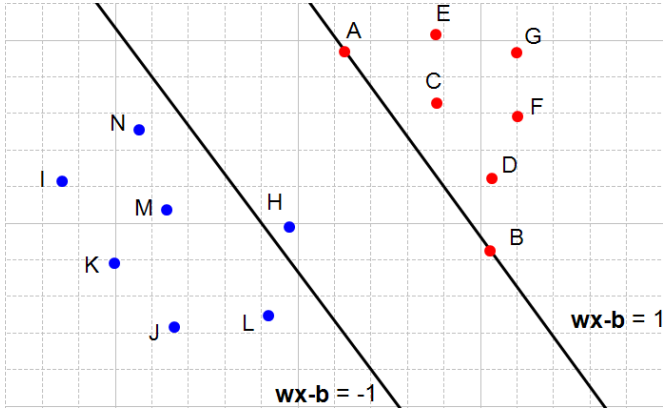
Understanding the constraints

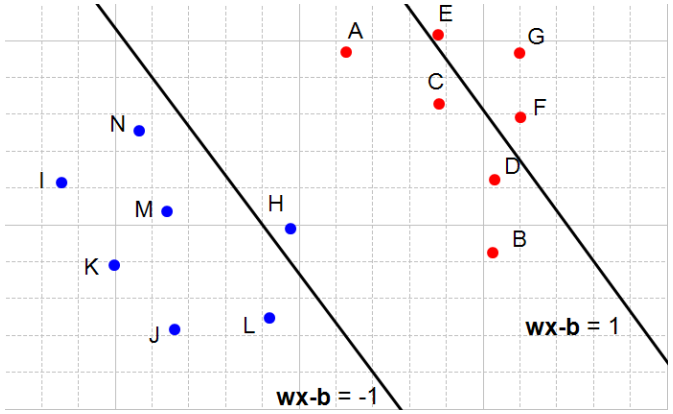




Understanding the constraints







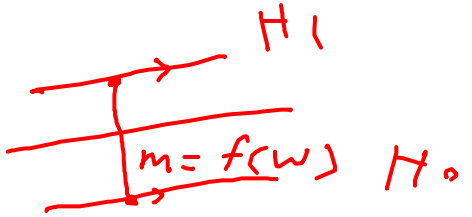
Combining both constraints

$-1(wx_i+b) \geq 1 \rightarrow wx_i+b \leq -1$

$$y_i(\mathbf{w} \cdot \mathbf{x_i} + b) \geq 1 \text{ for all } 1 \leq i \leq n$$

$$\begin{cases} wx_i+b \geq 1 & y_i = 1 \\ wx_i+b \leq -1 & y_i = -1 \end{cases}$$

Maximize the distance between the two hyperplanes



Finding the biggest margin, is the same thing as finding the optimal hyperplane.

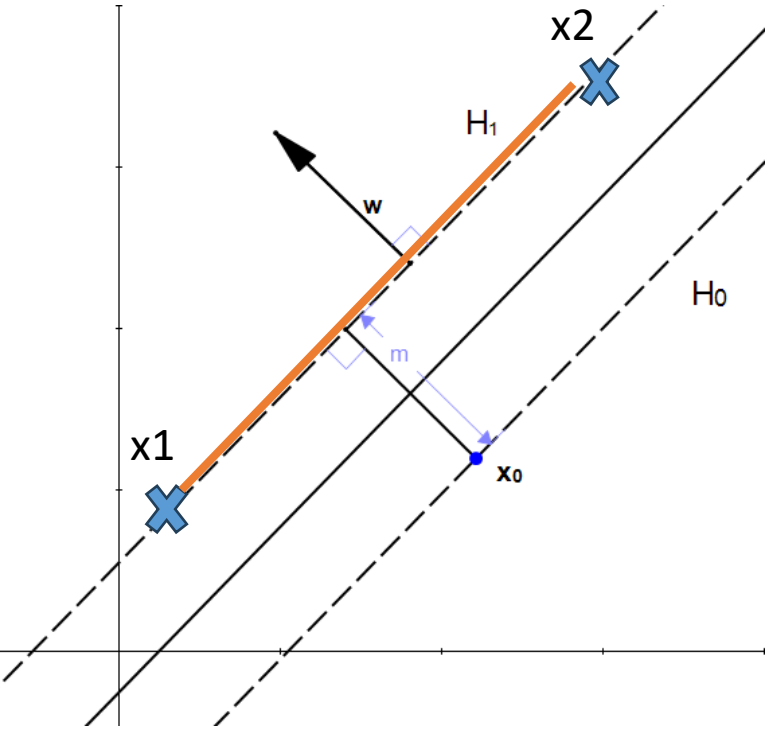
- **a) What is the distance between our two hyperplanes ?**

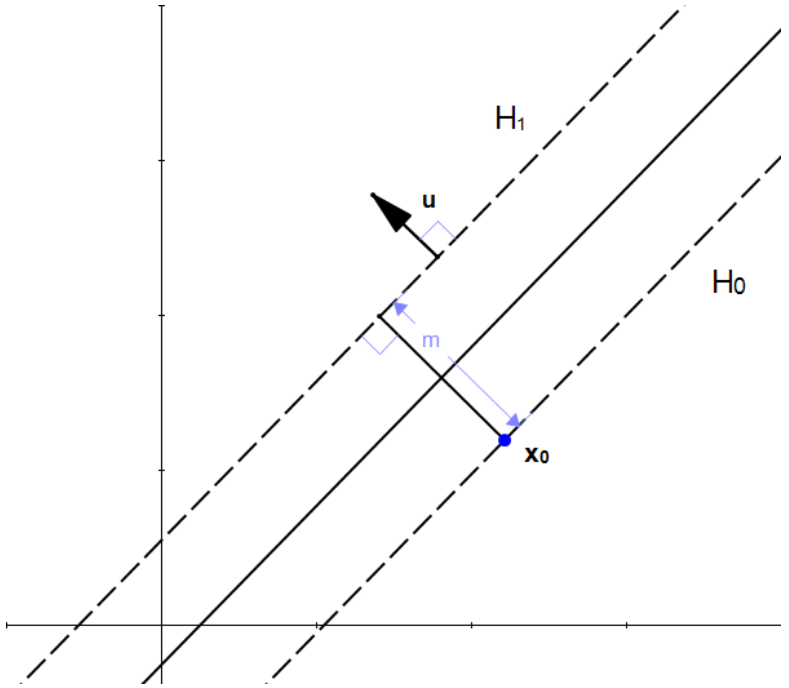
- $\mathcal{H}_0$ be the hyperplane having the equation $\mathbf{w} \cdot \mathbf{x} + b = -1$
- $\mathcal{H}_1$ be the hyperplane having the equation $\mathbf{w} \cdot \mathbf{x} + b = 1$
- $\mathbf{x}_0$ be a point in the hyperplane $\mathcal{H}_0$.

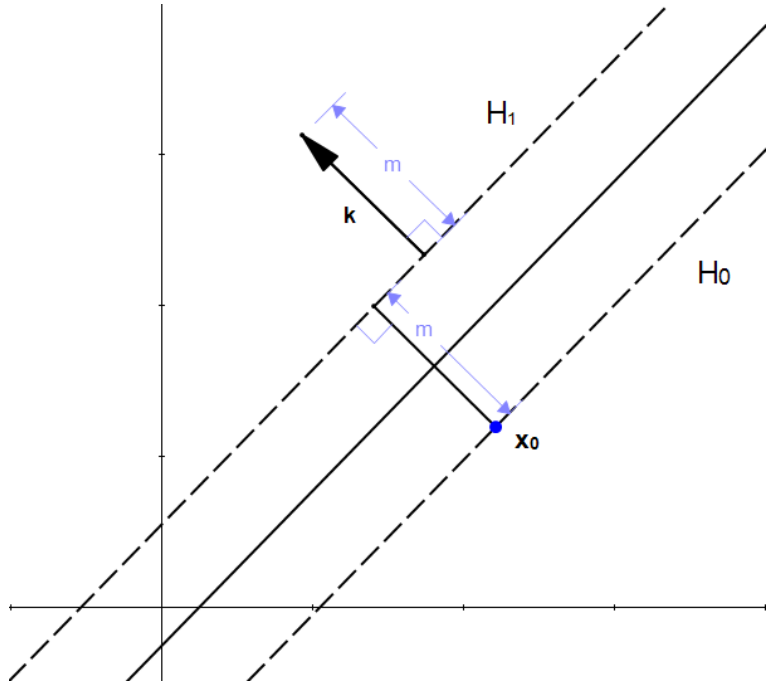
m is the distance between hyperplanes $\mathcal{H}_0$ and $\mathcal{H}_1$.









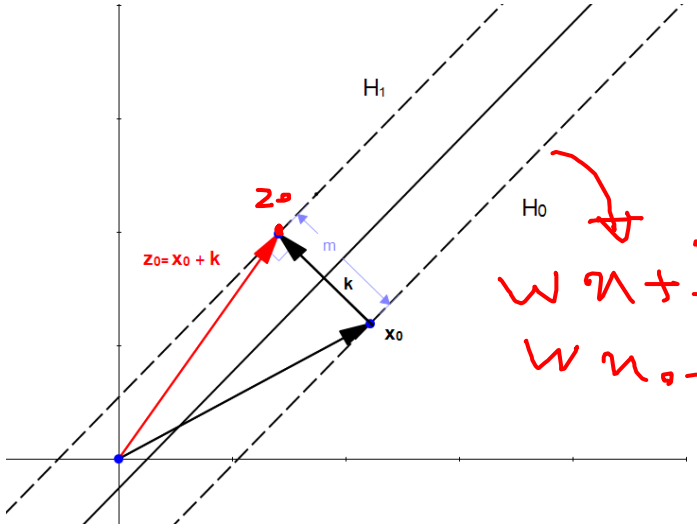


$$w.x_1 + b = 1$$
$$w.x_2 + b = 1$$
$$w.(x_2 - x_1) = 0$$

$$\mathbf{u} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

$\|u\|=1$

$$\mathbf{k} = m\mathbf{u} = m\frac{\mathbf{w}}{\|\mathbf{w}\|}$$



$w x_0 + b = -1$

$w x_0 + b = -1$

$$\mathbf{w} \cdot \mathbf{z}_0 + b = 1$$

$$\mathbf{w} \cdot (\mathbf{x}_0 + \mathbf{k}) + b = 1$$

$$\mathbf{w} \cdot (\mathbf{x}_0 + m\frac{\mathbf{w}}{\|\mathbf{w}\|}) + b = 1$$

$$\mathbf{w} \cdot \mathbf{x}_0 + m\frac{\mathbf{w} \cdot \mathbf{w}}{\|\mathbf{w}\|} + b = 1$$

$$w \cdot w = \|w\|^2$$

$$w = (w_1, w_2)$$

$$w \cdot w = w_1^2 + w_2^2$$

$$\|w\| = \sqrt{w_1^2 + w_2^2}$$

$$\mathbf{w} \cdot \mathbf{x}_0 + m\frac{\mathbf{w} \cdot \mathbf{w}}{\|\mathbf{w}\|} + b = 1$$

$$\mathbf{w} \cdot \mathbf{x}_0 + m\frac{\|\mathbf{w}\|^2}{\|\mathbf{w}\|} + b = 1$$

$$\mathbf{w} \cdot \mathbf{x}_0 + m\|\mathbf{w}\| + b = 1$$

$$\mathbf{w} \cdot \mathbf{x}_0 + b = 1 - m\|\mathbf{w}\|$$





$$\mathbf{w} \cdot \mathbf{x}_0 + b = 1 - m\|\mathbf{w}\|$$

$$\mathbf{w} \cdot \mathbf{x}_0 + b = -1$$

$$-1 = 1 - m\|\mathbf{w}\|$$

$$m\|\mathbf{w}\| = 2$$

$$m = \frac{2}{\|\mathbf{w}\|}$$

This is it ! We found a way to compute m.





Maximizing the margin is the same thing as minimizing the norm of w

$m = \frac{2}{\|w\|}$ ↑ ↓

Minimize in $(\mathbf{w}, b)$

$$\|\mathbf{w}\|$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x_i} + b) \geq 1$

(for any $i = 1, \ldots, n$)





Machine Learning

Support Vector Machine (SVM)

Part 4: Soft margin SVM

Dr. Mehran Safayani



safayani@iut.ac.ir

https://people.iut.ac.ir/en/safayani

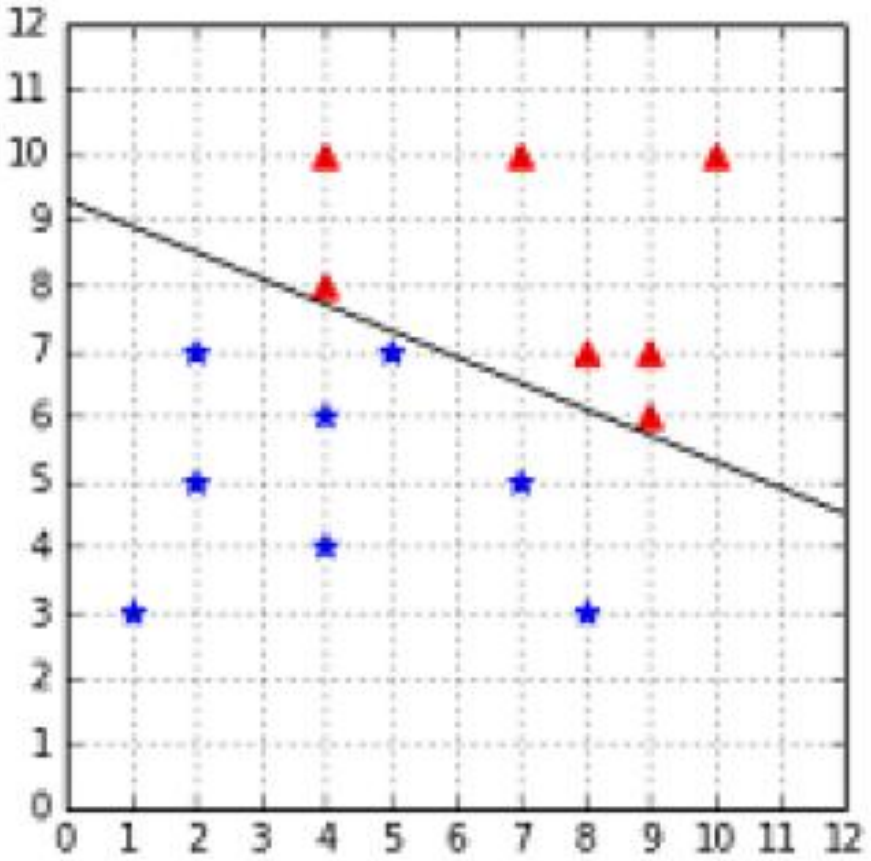
https://www.aparat.com/mehran.safayani

https://github.com/safayani/machine_learning_course

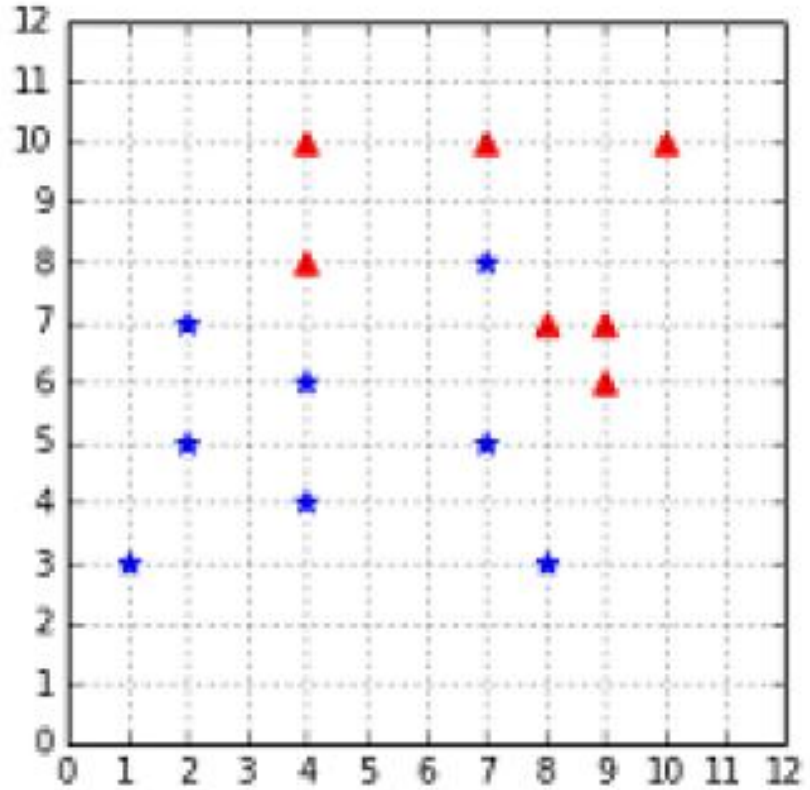
https://git.iut.ac.ir/safayani/courses/-/tree/main/Machine_Learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

- outlier data point at (5, 7)



- *The outlier at (7, 8) breaks linear separability*



Soft margin

- **Slack variables**

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \zeta_i$$

The problem is that we could choose a huge value of for every example, and all the constraints will be satisfied.





Soft margin SVM

$$\underset{\mathbf{w},b,\zeta}{\text{minimize}} \quad \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{subject to} \quad y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \zeta_i$$

$$\zeta_i \geq 0 \quad \text{for any } i = 1, \ldots, m$$





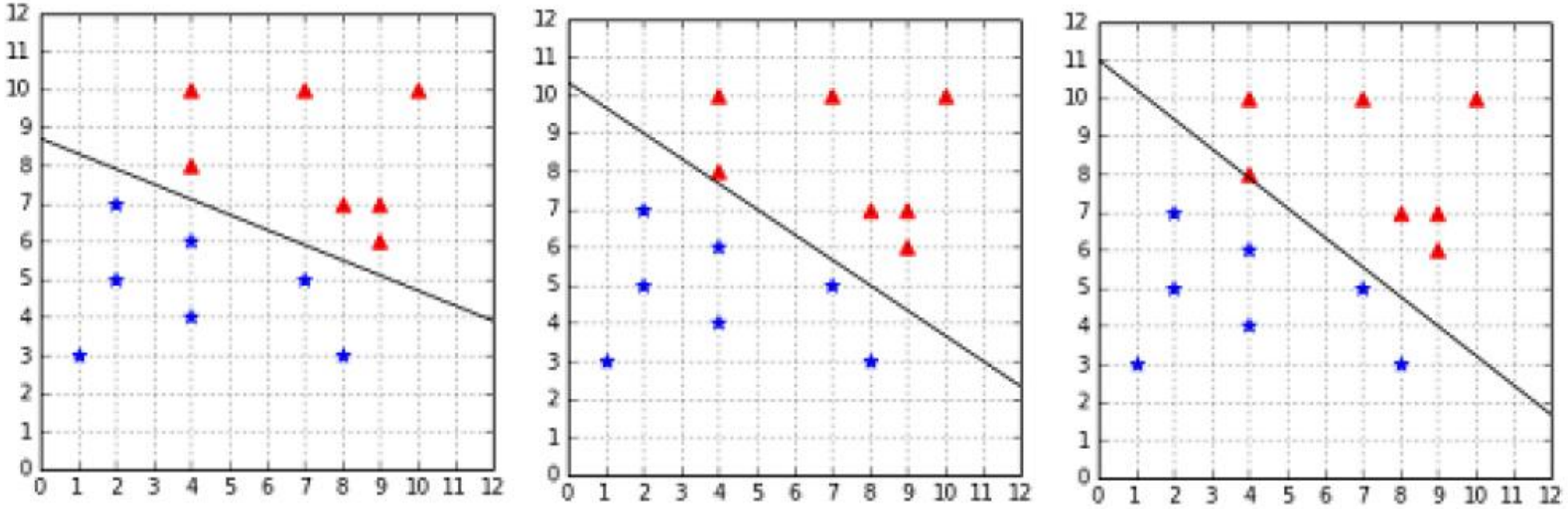
Soft margin SVM

$$\begin{aligned} \underset{\mathbf{w},b,\zeta}{\text{minimize}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}\zeta_i \\ \text{subject to} \quad & y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \zeta_i \\ & \zeta_i \geq 0 \quad \text{for any } i = 1, \ldots, m \end{aligned}$$





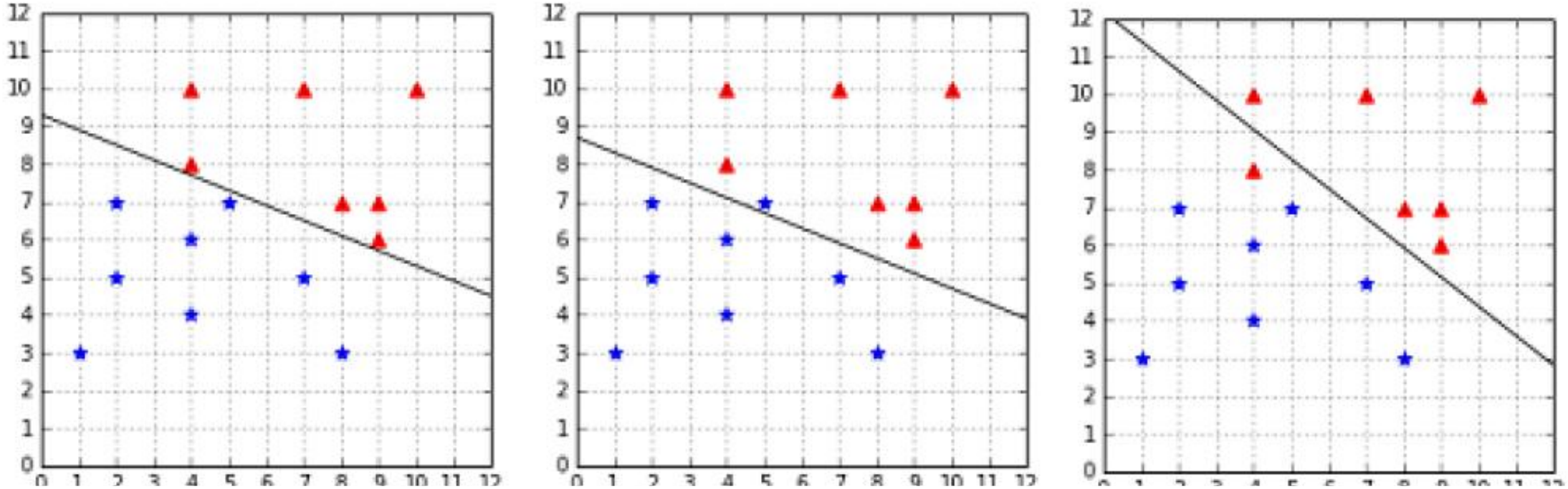
Understanding what C does



Effect of C=+Infinity, C=1, and C=0.01 on a linearly separable dataset

- When C is close to zero, there is basically no constraint anymore, and we end up with a hyperplane not classifying anything.
- It seems that when the data is linearly separable, sticking with a big C is the best choice

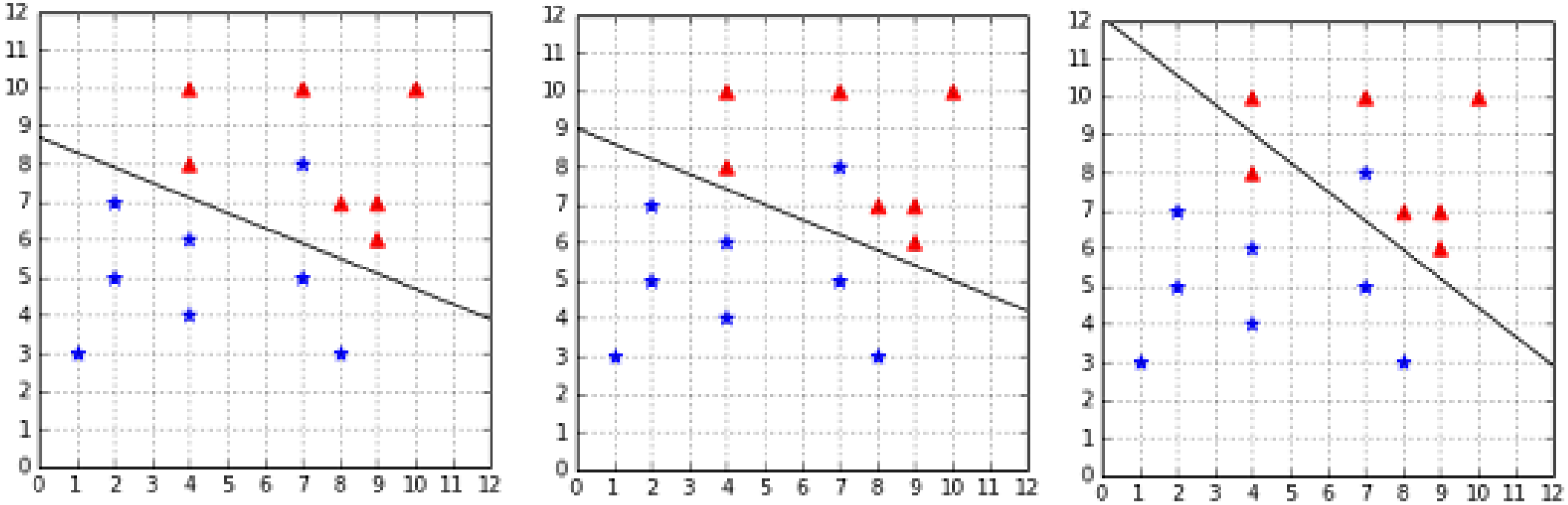
Understanding what C does



Effect of C=+Infinity, C=1, and C=0.01 on a linearly separable dataset with an outlier

when we use C=1, we end up with a hyperplane very close to the one of the hard margin classifier without outlier.

Understanding what C does



Effect of C=3, C=1, and C=0.01 on a non-separable dataset with an outlier

- we cannot use C=infinity because there is no solution meeting all the hard margin constraints.
the best hyperplane is achieved with C=3

Understanding what C does

Rules of thumb:

- A small C will give a wider margin, at the cost of some misclassifications.
- A huge C will give the hard margin classifier and tolerates zero constraint violation.
- The key is to find the value of C such that noisy data does not impact the solution too much.





Machine Learning

Support Vector Machine (SVM)

Part 5: Kernel

Dr. Mehran Safayani



safayani@iut.ac.ir

https://people.iut.ac.ir/en/safayani

https://www.aparat.com/mehran.safayani

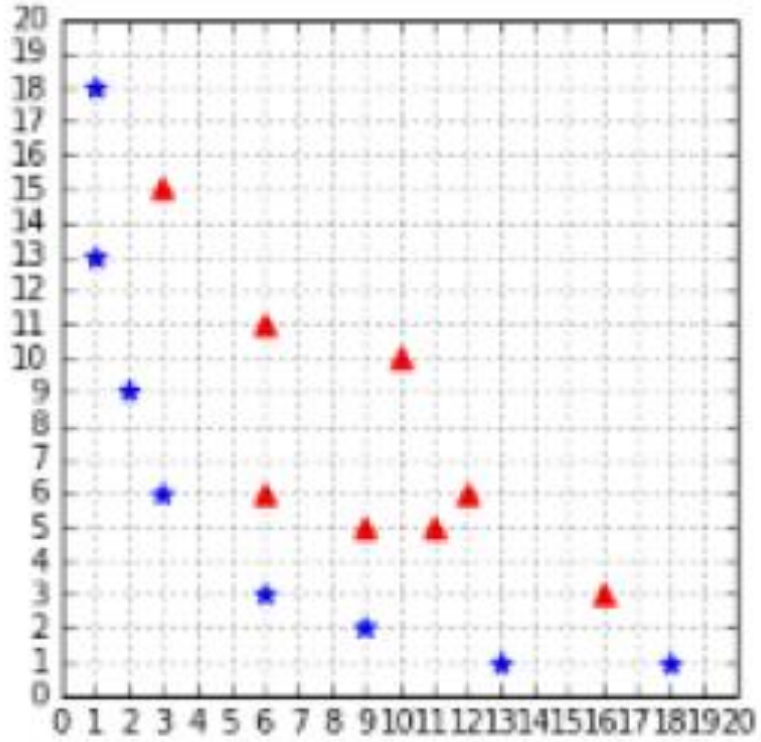
https://github.com/safayani/machine_learning_course

https://git.iut.ac.ir/safayani/courses/-/tree/main/Machine_Learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

Kernels

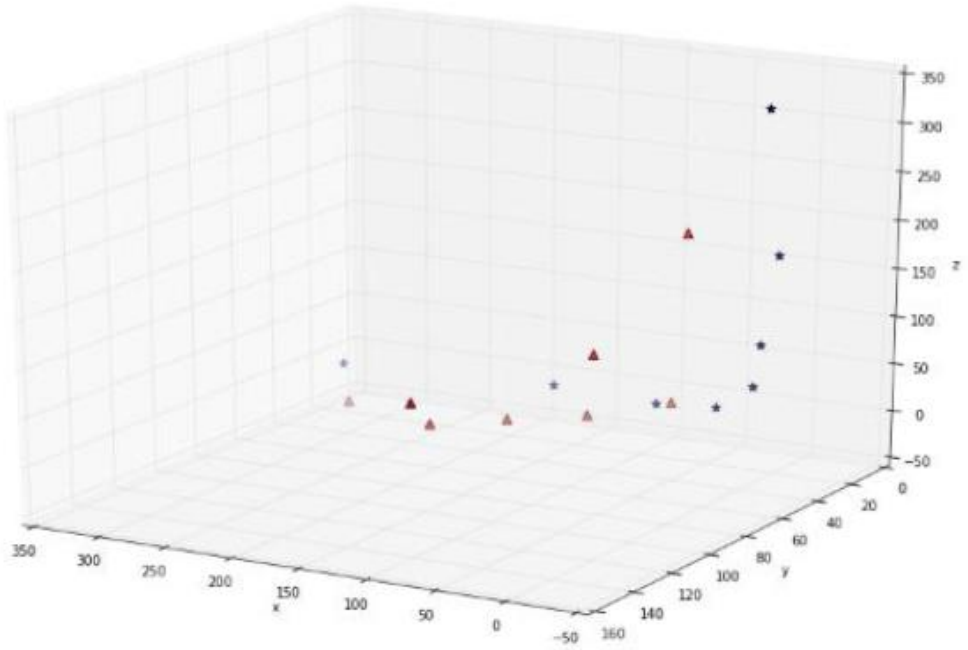
- **Can we classify non-linearly separable data?**

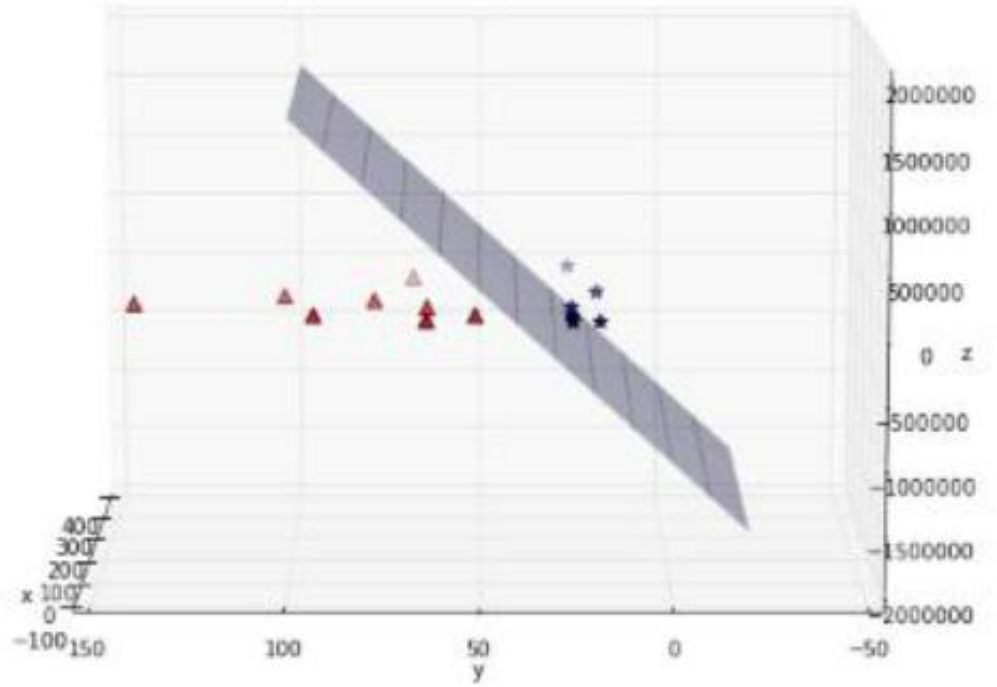


A straight line cannot separate the data

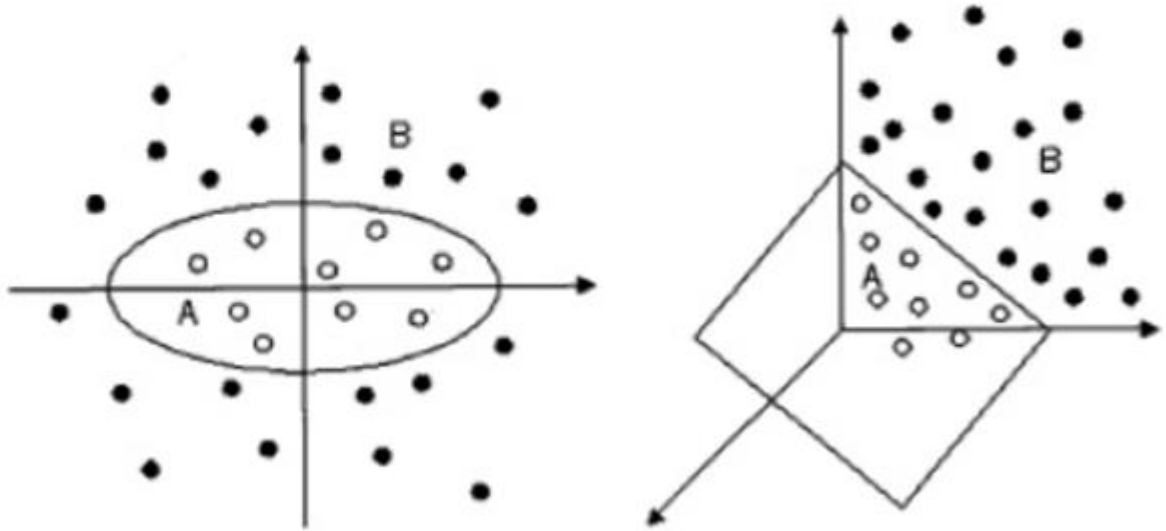
polynomial mapping

$$\phi(x_1, x_2) = (x_1^2,\ \sqrt{2}x_1x_2,\ x_2^2)$$





$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$
$$(x_1, x_2) \rightarrow (z_1, z_2, z_3) := (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$



Basic recipe

1. Transform every two-dimensional vector into a three-dimensional vector using the transform method (quadratic mapping)

2. Train the SVMs using the 3D dataset.

3. For each new example we wish to predict, transform it using the **transform** method before passing it to the **predict** method





One of the main drawbacks of the previous method is that we must transform every example. If we have millions or billions of examples and that transform method is complex, that can take a huge amount of time. This is when kernels come to the rescue.





Kernel Trick

- Kernel Function: $K(x_i, x_j) = \langle \phi(x_i) \cdot \phi(x_j) \rangle$

$$\vec{x_i} = (x_{i1}, x_{i2}) \quad \Rightarrow \quad \phi(\vec{x_i}) = (x_{i1}^2, \sqrt{2}x_{i1}x_{i2}, x_{i2}^2)$$

$$\vec{x_j} = (x_{j1}, x_{j2}) \quad \Rightarrow \quad \phi(\vec{x_j}) = (x_{j1}^2, \sqrt{2}x_{j1}x_{j2}, x_{j2}^2)$$

$$K(\vec{x_i}, \vec{x_j}) = x_{i1}^2 \times x_{j1}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2} + x_{i2}^2 x_{j2}^2$$





Examples Kernel Trick

$\vec{x} = (x_1, x_2)$

$\vec{z} = (z_1, z_2)$

$K(x, z) = \langle \vec{x} \cdot \vec{z} \rangle^2$

$$K(x_1, x_2) = \langle \phi(x_i) \cdot \phi(x_j) \rangle$$

$$\begin{aligned} K(x,z) &= \langle \vec{x} \cdot \vec{z} \rangle^2 \\ &= (x_1 z_1 + x_2 z_2)^2 \\ &= (x_1^2 z_1^2 + 2 x_1 z_1 x_2 z_2 + x_2^2 z_2^2) \\ &= \left\langle (x_1^2, \sqrt{2} x_1 x_2, x_2^2) \cdot (z_1^2, \sqrt{2} z_1 z_2, z_2^2) \right\rangle \\ &= \langle \phi(\vec{x}) \cdot \phi(\vec{z}) \rangle \end{aligned}$$

3 multiplications

4+4+3=11 multiplications

mapping function ϕ fused in K

$\rightarrow$ implicit $\phi(\vec{x}) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$

ϕ not necessary any more

possible to operate in any n-dimensional FS

complexity independent of FS





Typical Kernels

- **Linear Kernel**

$$K(x, z) = x \,.\, z'$$

- **Polynomial Kernel:**

$$K(x, z) = \left(\langle x \cdot z \rangle + \theta\right)^d \text{ for } d \geq 0,$$

- **Radial Basis Function (Gaussian Kernel):**

$$K(x, z) = e^{-\frac{||x-z||^2}{2\sigma^2}}$$

- **(Sigmoid Kernel):**

$$K(x, z) = \tanh(\eta\langle x \cdot z \rangle + \theta)$$

- **Inverse multi-quadric:**

$$K(x, z) = \frac{1}{\sqrt{||\ x - z\ ||^2\ 2\sigma^2 + c^2}}$$





Example: Gaussian kernel

$$k(x,y) = \exp\left(-\frac{1}{2}(x-y)^2\right) \qquad k(x,y) = \exp\left(-\frac{1}{2}(x^2+y^2-2xy)\right)$$

$$k(x,y) = \exp\left(-\frac{1}{2}x^2\right) \cdot \exp\left(-\frac{1}{2}y^2\right) \cdot \exp(xy)$$

$$e^z = \sum_{n=0}^{\infty} \frac{z^n}{n!} = 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \cdots$$

the Taylor series expands an exponential function into an **infinite sum**:

$$\exp(xy) = 1 + xy + \frac{x^2y^2}{2!} + \frac{x^3y^3}{3!} + \cdots \qquad k(x,y) = \exp\left(-\frac{1}{2}x^2\right)\exp\left(-\frac{1}{2}y^2\right)\left[1 + xy + \frac{x^2y^2}{2!} + \cdots\right]$$

$$\phi(x)^T\phi(y) \qquad \phi(x) = \exp\left(-\frac{1}{2}x^2\right)\begin{bmatrix} 1 \\ x/\sqrt{1!} \\ x^2/\sqrt{2!} \\ \vdots \\ x^n/\sqrt{n!} \\ \vdots \end{bmatrix}$$





Kernel functions

Kernel functions must be continuous, symmetric, and most preferably should have a positive (semi-) definite Gram matrix.

$$G(x_1, \ldots, x_n) = \begin{vmatrix} \langle x_1, x_1 \rangle & \langle x_1, x_2 \rangle & \ldots & \langle x_1, x_n \rangle \\ \langle x_2, x_1 \rangle & \langle x_2, x_2 \rangle & \ldots & \langle x_2, x_n \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle x_n, x_1 \rangle & \langle x_n, x_2 \rangle & \ldots & \langle x_n, x_n \rangle \end{vmatrix}.$$

$$\alpha^T G \alpha = \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j G_{i,j} \geq 0$$

$K(x, y) = x^T y$

$K_{i,j} = x_i^T x_j$

$$\sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j (x_i^T x_j) = \left(\sum_{i=1}^{n} \alpha_i x_i \right)^T \left(\sum_{j=1}^{n} \alpha_j x_j \right) = \left\| \sum_{i=1}^{n} \alpha_i x_i \right\|^2 \geq 0$$

Kernel SVM:

Maximizing the margin is the same thing as minimizing the norm of w

$$\begin{aligned} \underset{\mathbf{w},b,\zeta}{\text{minimize}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}\zeta_i \\ \text{subject to} \quad & y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \zeta_i \\ & \zeta_i \geq 0 \quad \text{for any } i = 1, \ldots, m \end{aligned}$$





Using kernels in SVM

$$\mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i \qquad b = \frac{1}{S} \sum_{i=1}^{S} (y_i - \mathbf{w} \cdot \mathbf{x}_i)$$

$$\begin{aligned} \underset{\alpha}{\text{maximize}} \quad & \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \\ \text{subject to} \quad & 0 \le \alpha_i \le C, \text{ for any } i = 1, \ldots, m \\ & \sum_{i=1}^{m} \alpha_i y_i = 0 \end{aligned}$$

$\phi(x_i) \cdot \phi(x_j)$

$K(x_i, x_j)$





Using kernels in SVM

$$\begin{aligned}
\underset{\alpha}{\text{maximize}} \quad & \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\
\text{subject to} \quad & 0 \leq \alpha_i \leq C, \text{ for any } i = 1, \ldots, m \\
& \sum_{i=1}^{m} \alpha_i y_i = 0
\end{aligned}$$





how to classify

Linear SVM ←

$$h(\mathbf{x}_i) = \text{sign}(\mathbf{w} \cdot \mathbf{x}_i + b) \quad > 0$$

$$\mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

$$h(\mathbf{x}_i) = \text{sign}\left(\sum_{j=1}^{S} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_i) + b \right)$$

$\phi(x_j) \cdot \phi(x_i)$

$K(x_i, x_j)$

K SVM ←

$$h(\mathbf{x}_i) = \text{sign}\left(\sum_{j=1}^{S} \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i) + b \right)$$





$$h(\mathbf{x}_i) = \text{sign}\left(\sum_{j=1}^{S} \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i) + b \right)$$

$\alpha_j y_j = \theta_j$

$$K(x, z) = e^{-\frac{\|x-z\|^2}{2\sigma^2}}$$

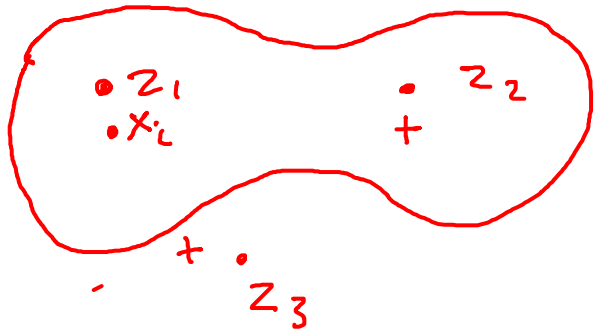
$K(x,z) = K(z,x)$

$\theta_1 f_1 + \theta_2 f_2 + \theta_3 f_3 + b$

$K(x, z_1) = f_1$

$K(x, z_2) = f_2$

$K(x, z_3) = f_3$



$\theta_1 = 1 \quad \theta_2 = 1 \quad \theta_3 = -1 \quad b = -0.5$

$f_1 \approx 1 \quad f_2 \approx 0 \quad f_3 \approx 0$

$\Rightarrow 1 + 1\times 0 + (-1)\times 0 - 0.5 > 0$

$f_1 = 0 \quad f_2 = 1 \quad f_3 = 0 \quad b = 0.5$

$1\times 0 + 1\times 1 + -1\times 0 - 0.5 = 1 > 0$

$f_1 = 0 \; ; \quad f_2 = 0 \; ; \quad f_3 = 1$

$1\times 0 + 1\times 0 + -1\times 1 - 0.5 = -1.5 < 0$

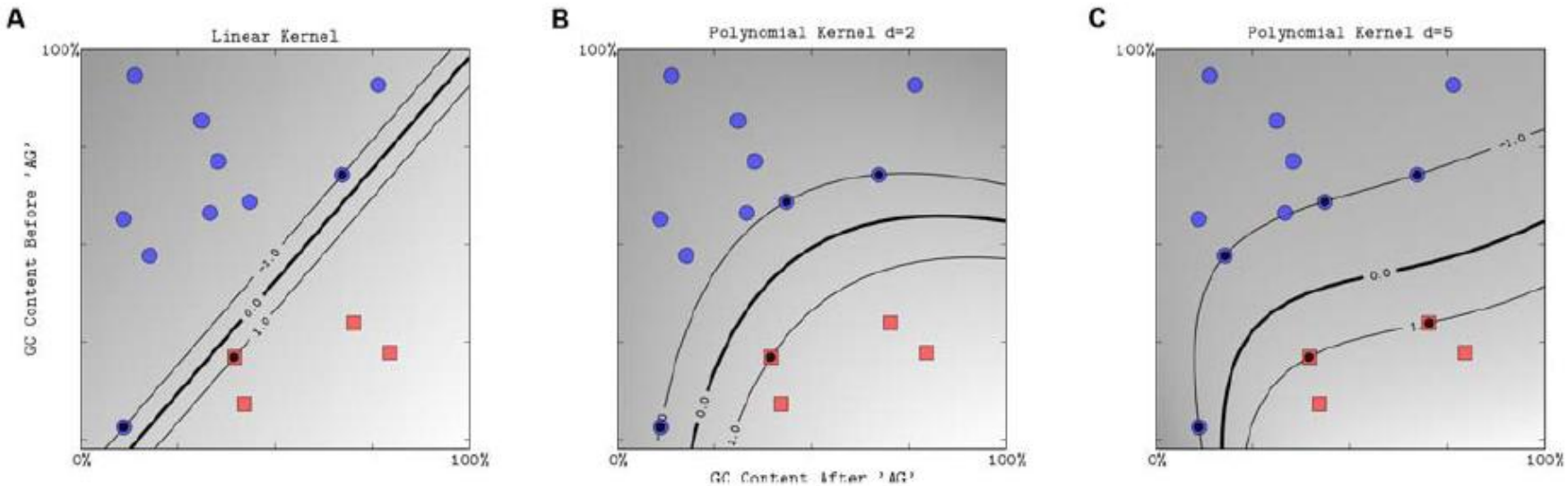


Figure 6. The effect of the degree of a polynomial kernel. The polynomial kernel of degree 1 leads to a linear separation (A). Higher-degree polynomial kernels allow a more flexible decision boundary (B,C). The style follows that of Figure 3.
doi:10.1371/journal.pcbi.1000173.g006

Ben-Hur A, Ong CS, Sonnenburg S, Schölkopf B, Rätsch G (2008) Support Vector Machines and Kernels for Computational Biology. PLoS Comput Biol 4(10): e1000173. https://doi.org/10.1371/journal.pcbi.1000173

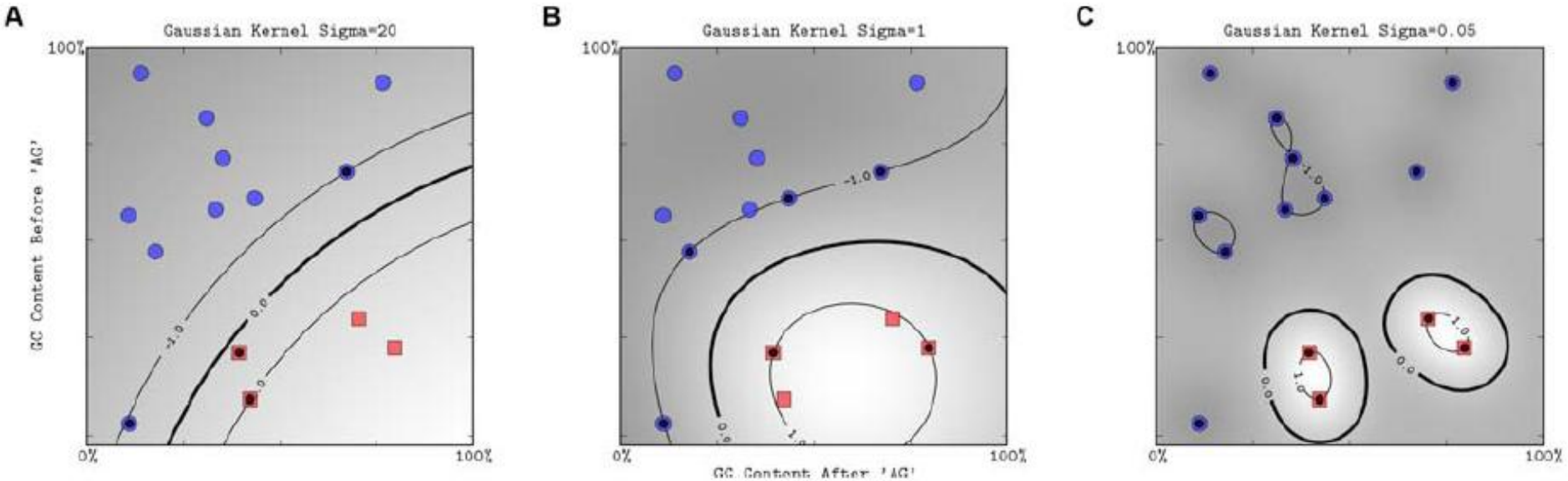


Figure 7. The effect of the width parameter of the Gaussian kernel (σ) for a fixed value of the soft-margin constant. For large values of σ (A), the decision boundary is nearly linear. As σ decreases, the flexibility of the decision boundary increases (B). Small values of σ lead to overfitting (C). The figure style follows that of Figure 3.
doi:10.1371/journal.pcbi.1000173.g007

Ben-Hur A, Ong CS, Sonnenburg S, Schölkopf B, Rätsch G (2008) Support Vector Machines and Kernels for Computational Biology. PLoS Comput Biol 4(10): e1000173. https://doi.org/10.1371/journal.pcbi.1000173

Machine Learning

Support Vector Machine (SVM)

Part 6: Optimization

Dr. Mehran Safayani



safayani@iut.ac.ir

https://people.iut.ac.ir/en/safayani

https://www.aparat.com/mehran.safayani

https://github.com/safayani/machine_learning_course

https://git.iut.ac.ir/safayani/courses/-/tree/main/Machine_Learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

Affine and convex functions

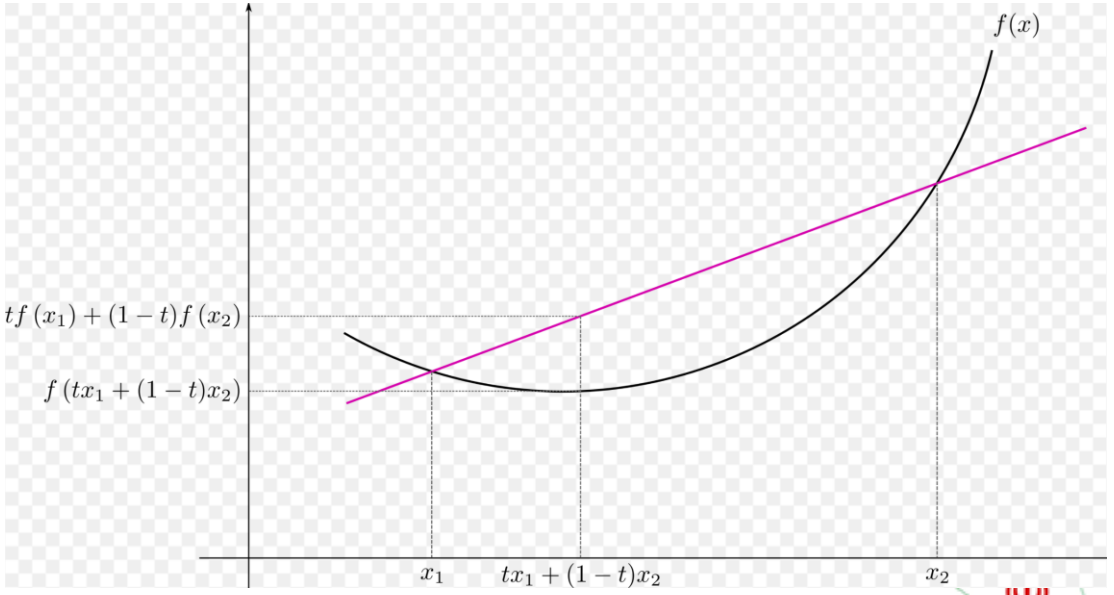
- Affine function

$$\mathbf{y} = f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}.$$

- Convex function:

For all $0 \leq t \leq 1$ and all $x_1, x_2 \in X$:

$$f\left(tx_1 + (1-t)x_2\right) \leq tf\left(x_1\right) + (1-t)f\left(x_2\right)$$



$$\begin{aligned} \min_w \quad & f(w) \\ \text{s.t.} \quad & g_i(w) \leq 0, \quad i = 1, \ldots, k \\ & h_i(w) = 0, \quad i = 1, \ldots, l. \end{aligned}$$

generalized Lagrangian

$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_{i=1}^{k} \alpha_i g_i(w) + \sum_{i=1}^{l} \beta_i h_i(w).$$

Here, the α_i's and β_i's are the Lagrange multipliers.





$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_{i=1}^{k} \alpha_i g_i(w) + \sum_{i=1}^{l} \beta_i h_i(w).$$

$$\theta_{\mathcal{P}}(w) = \max_{\alpha, \beta : \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta).$$

Here, the "$\mathcal{P}$" subscript stands for "primal."

$$\theta_{\mathcal{P}}(w) = \begin{cases} f(w) & \text{if } w \text{ satisfies primal constraints} \\ \infty & \text{otherwise.} \end{cases}$$





$$\min_{w} \theta_{\mathcal{P}}(w) = \min_{w} \max_{\alpha,\beta\,:\,\alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta),$$

Now, let's look at a slightly different problem. We define

$$\theta_{\mathcal{D}}(\alpha, \beta) = \min_{w} \mathcal{L}(w, \alpha, \beta).$$

We can now pose the **dual** optimization problem:

$$\max_{\alpha,\beta\,:\,\alpha_i \geq 0} \theta_{\mathcal{D}}(\alpha, \beta) = \max_{\alpha,\beta\,:\,\alpha_i \geq 0} \min_{w} \mathcal{L}(w, \alpha, \beta).$$





$$d^* = \max_{\alpha,\beta\,:\,\alpha_i \geq 0} \min_{w} \mathcal{L}(w, \alpha, \beta) \leq \min_{w} \max_{\alpha,\beta\,:\,\alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta) = p^*.$$

$$d^* = p^*,$$

Suppose f and the g_i's are convex,[6] and the h_i's are affine.[7] Suppose further that the constraints g_i are (strictly) feasible; this means that there exists some w so that $g_i(w) < 0$ for all i.





Given a real valued function $P(x,y) : X \times Y \to \mathbb{R}$ one has

$$\inf_{x\in X} \sup_{y\in Y} P(x,y) \geq \sup_{y\in Y} \inf_{x\in X} P(x,y)$$

- Proof

$$\sup_y P(x',y) \geq P(x',y') \geq \inf_x P(x,y') \quad \forall x', y'$$

$$\sup_y P(x',y) \geq \inf_x P(x,y') \quad \forall x', y'$$

$$\sup_y P(x',y) \geq \sup_y \inf_x P(x,y) \quad \forall x'$$

$$\inf_x \sup_y P(x,y) \geq \sup_y \inf_x P(x,y)$$





KKT conditions

Under our above assumptions, there must exist w^*, α^*, β^* so that w^* is the solution to the primal problem, α^*, β^* are the solution to the dual problem, and moreover $p^* = d^* = \mathcal{L}(w^*, \alpha^*, \beta^*)$. Moreover, w^*, α^* and β^* satisfy the **Karush-Kuhn-Tucker (KKT) conditions**, which are as follows:

$$
\begin{aligned}
\frac{\partial}{\partial w_i}\mathcal{L}(w^*, \alpha^*, \beta^*) &= 0, \quad i = 1, \ldots, d \\
\frac{\partial}{\partial \beta_i}\mathcal{L}(w^*, \alpha^*, \beta^*) &= 0, \quad i = 1, \ldots, l \\
\alpha_i^* g_i(w^*) &= 0, \quad i = 1, \ldots, k \\
g_i(w^*) &\leq 0, \quad i = 1, \ldots, k \\
\alpha^* &\geq 0, \quad i = 1, \ldots, k
\end{aligned}
$$

Moreover, if some w^*, α^*, β^* satisfy the KKT conditions, then it is also a solution to the primal and dual problems.



$$\min_{w,b} \quad \frac{1}{2}||w||^2$$
$$\text{s.t.} \quad y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad i = 1, \ldots, n$$

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i \left[y^{(i)}(w^T x^{(i)} + b) - 1 \right].$$

$$\nabla_w \mathcal{L}(w, b, \alpha) = w - \sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)} = 0$$

$$w = \sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)}.$$

$$\frac{\partial}{\partial b} \mathcal{L}(w, b, \alpha) = \sum_{i=1}^{n} \alpha_i y^{(i)} = 0.$$





$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j (x^{(i)})^T x^{(j)} - b \sum_{i=1}^{n} \alpha_i y^{(i)}.$$

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j (x^{(i)})^T x^{(j)}.$$





$$\max_\alpha \quad W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle.$$

$$\text{s.t.} \quad \alpha_i \geq 0, \quad i = 1, \ldots, n$$

$$\sum_{i=1}^{n} \alpha_i y^{(i)} = 0,$$

Sequential minimal optimization

$$\begin{aligned} w^T x + b &= \left(\sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)} \right)^T x + b \\ &= \sum_{i=1}^{n} \alpha_i y^{(i)} \langle x^{(i)}, x \rangle + b. \end{aligned}$$

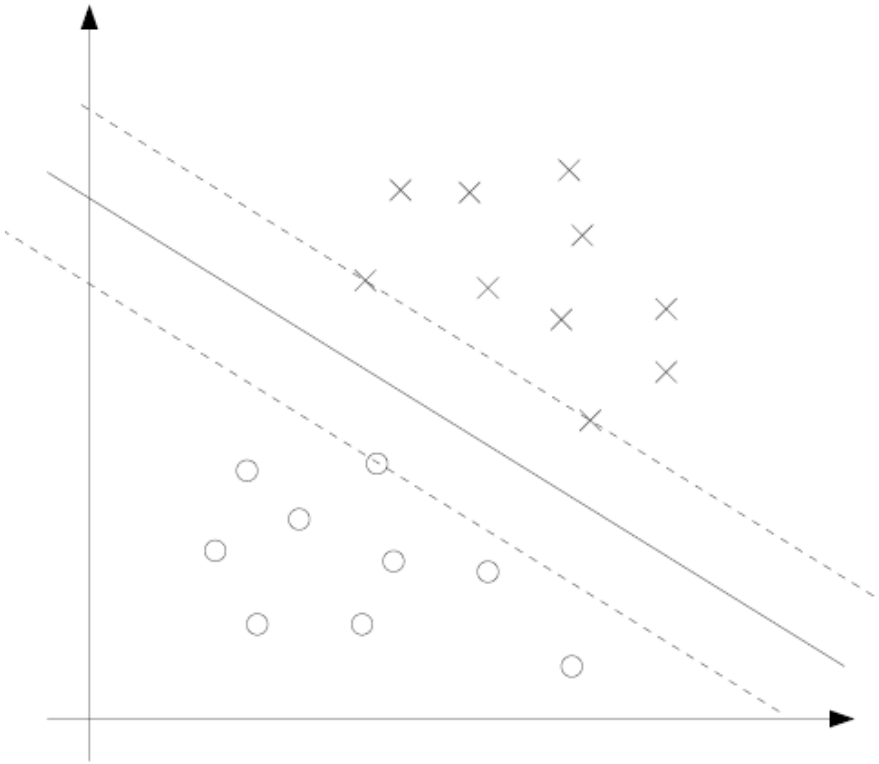




Karush-Kuhn-Tucker (KKT) conditions, which are as follows:

$$\begin{aligned}
\frac{\partial}{\partial w_i}\mathcal{L}(w^*, \alpha^*, \beta^*) &= 0, \quad i = 1, \ldots, d \\
\alpha_i^* g_i(w^*) &= 0, \quad i = 1, \ldots, k \\
g_i(w^*) &\leq 0, \quad i = 1, \ldots, k \\
\alpha^* &\geq 0, \quad i = 1, \ldots, k
\end{aligned}$$

$$g_i(w) = -y^{(i)}(w^T x^{(i)} + b) + 1 \leq 0.$$



support vectors

$$\underset{\mathbf{w},b,\zeta}{\text{minimize}} \quad \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}\zeta_i$$

$$\text{subject to} \quad y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \zeta_i$$

$$\zeta_i \geq 0 \quad \text{for any } i = 1, \ldots, m$$





$$\mathcal{L}(w, b, \zeta, \alpha, r) = \frac{1}{2} w^T w + C \sum_{i=1}^{n} \zeta_i - \sum_{i=1}^{n} \alpha_i \left[y^{(i)}(x^T w + b) - 1 + \zeta_i \right] - \sum_{i=1}^{n} r_i \zeta_i$$

$$w = \sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)}. \qquad \sum_{i=1}^{n} \alpha_i y^{(i)} = 0. \qquad \alpha = C1 - r$$

Complementary conditions:

$\alpha_i \left[y^{(i)}(x^T w + b) - 1 + \zeta_i \right] = 0$ $\quad r_i \zeta_i = 0$

Hence at optimality we have $w = \sum_{1}^{n} \alpha_i y^{(i)} x^{(i)}$. ., and ' α_i is nonzero only if $\alpha_i \left[y^{(i)}(x^T w + b) - 1 + \zeta_i \right] = 0$

Such points i are called support points

- For support point i, if $\zeta_i = 0$, then x_i lies on edge of margin, and $\alpha_i \in (0, C]$;
- For support point i, if $\zeta_i \neq 0$, then x_i lies on wrong side of margin, and $\alpha_i = C$





$$\mathcal{L}(w, b, \xi, \alpha, r) = \frac{1}{2} w^T w + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \alpha_i \left[y^{(i)}(x^T w + b) - 1 + \xi_i \right] - \sum_{i=1}^{n} r_i \xi_i.$$

$$w = \sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)}. \qquad \sum_{i=1}^{n} \alpha_i y^{(i)} = 0. \qquad \alpha = C1 - r$$

Complementary conditions:

$\alpha_i \left[y^{(i)}(x^T w + b) - 1 + \xi_i \right] = 0$ $\qquad$ $r_i \xi_i = 0$

Hence at optimality we have $w = \sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)}$., and α_i is nonzero only if $\left[y^{(i)}(x^T w + b) - 1 + \xi_i \right] = 0$ Such points i are called support points

- For support point i, if $\xi_i = 0$, then x_i lies on edge of margin, and $\alpha_i \in (0, C]$;
- For support point i, if $\xi_i \neq 0$, then x_i lies on wrong side of margin, and $\alpha_i = C$

$$
\begin{aligned}
\max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle \\
\text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \ldots, n \\
& \sum_{i=1}^{n} \alpha_i y^{(i)} = 0,
\end{aligned}
$$

the KKT dual-complementarity conditions

$$
\begin{aligned}
\alpha_i = 0 \quad &\Rightarrow \quad y^{(i)}(w^T x^{(i)} + b) > 1 \\
\alpha_i = C \quad &\Rightarrow \quad y^{(i)}(w^T x^{(i)} + b) < 1 \\
0 < \alpha_i < C \quad &\Rightarrow \quad y^{(i)}(w^T x^{(i)} + b) = 1.
\end{aligned}
$$





Machine Learning

Support Vector Machine (SVM)

Part 7: Advanced topics: SGD Optimization, SVR, and Logistic Connections

Dr. Mehran Safayani



safayani@iut.ac.ir

https://people.iut.ac.ir/en/safayani

https://www.aparat.com/mehran.safayani

https://github.com/safayani/machine_learning_course

https://git.iut.ac.ir/safayani/courses/-/tree/main/Machine_Learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

Machine Learning

Support Vector Machine (SVM)

Part VII: Advanced topics: SGD Optimization, SVR, and Logistic Connections

Dr. Mehran Safayani

safayani@iut.ac.ir

safayani.iut.ac.ir



https://www.aparat.com/mehran.safayani

https://github.com/safayani/machine_learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

Optimizing Soft-Margin SVM with SGD

$$\begin{aligned} \underset{\mathbf{w},b,\zeta}{\text{minimize}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}\zeta_i \\ \text{subject to} \quad & y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \zeta_i \\ & \zeta_i \geq 0 \quad \text{for any } i = 1,\ldots,m \end{aligned}$$

$\hat{y}_i = \mathbf{w}^T\mathbf{x}_i + b \qquad y_i \in \{-1, 1\}$

$if\ y_i\hat{y}_i \geq 1\ \ then\ \ \zeta_i = 0\ \ \rightarrow y_i\hat{y}_i \geq 1 - \zeta_i$

$if\ y_i\hat{y}_i < 1\ \ then\ \ \zeta_i = 1 - y_i\hat{y}_i\ \ \rightarrow y_i\hat{y}_i \geq 1 - (1 - y_i\hat{y}_i) = y_i\hat{y}_i$ for example $y_i\hat{y}_i = -2\ \ \zeta_i = 3$

Hinge Loss function $\zeta_i = \max(0, 1 - y_i\hat{y}_i)$

$$L(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\max(0, 1 - y_i\hat{y}_i) = \lambda\|w\|^2 + \sum_{i=1}^{N}\max(0,1 - y_i\hat{y}_i)$$

$\lambda = \frac{1}{2C}$





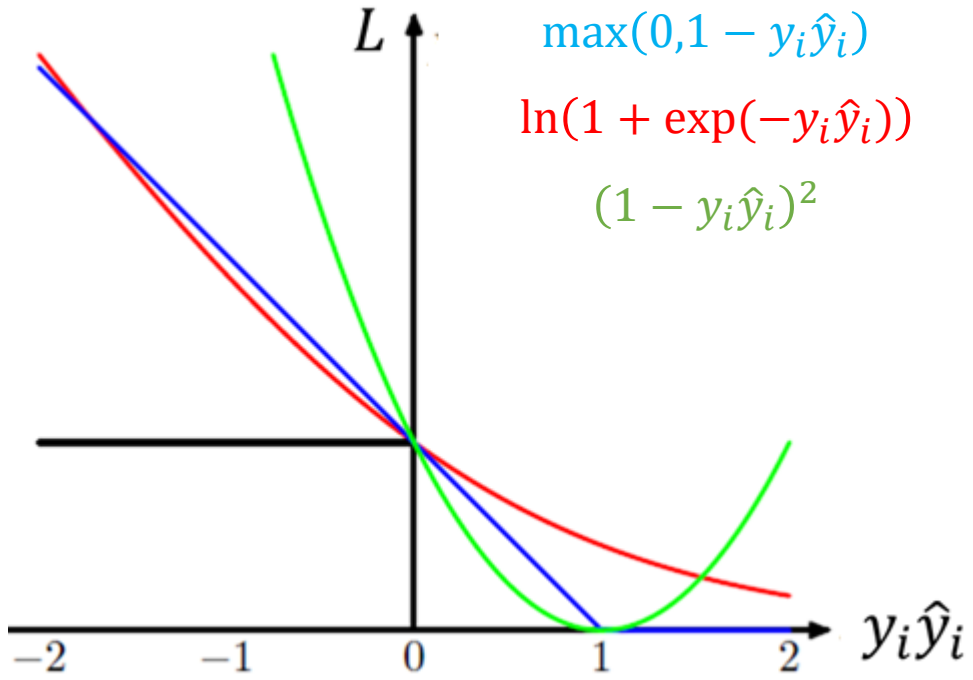
SVM as Logistic regression

- SVM loss

$$L_{svm}(w,b) = \lambda\|w\|^2 + \sum_{i=1}^{N} \max(0,1 - y_i\hat{y}_i)$$

- Logistic regression loss

$$L_{LR}(w,b) = \lambda\|w\|^2 + \sum_{i=1}^{N} \ln(1 + \exp(-y_i\hat{y}_i))$$



Plot of the 'hinge' error function used in support vector machines, shown in blue, along with the error function for logistic regression, rescaled by a factor of $1/\ln(2)$ so that it passes through the point $(0, 1)$, shown in red. Also shown are the misclassification error in black and the squared error in green.

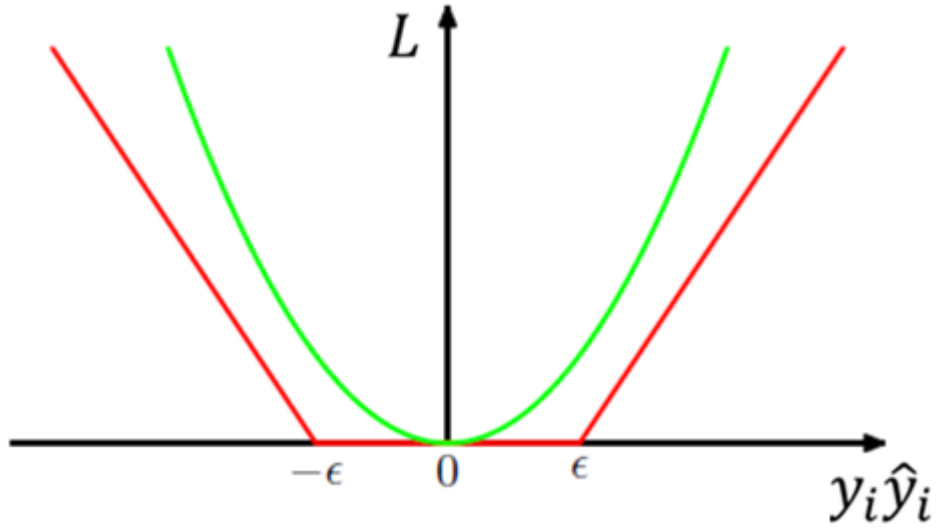
Support vector for Regression (SVR)

- Standard Regularized Regression (Ridge Regression)

$$\frac{1}{2}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

- The ϵ-Insensitive Loss Function

$$E_\epsilon(\hat{y}_i - y_i) = \begin{cases} 0, & \text{if } |\hat{y}_i - y_i| \leqslant \epsilon \\ |\hat{y}_i - y_i| - \epsilon, & \text{otherwise} \end{cases} \qquad E_\epsilon = \max(0, |\hat{y}_i - y_i| - \epsilon)$$



Plot of an ϵ-insensitive error function (in red) in which the error increases linearly with distance beyond the insensitive region. Also shown for comparison is the quadratic error function (in green).

- The Final SVR Objective Function

$$C\sum_{i=1}^{N} E_\epsilon(\hat{y}_i - y_i) + \frac{1}{2}\|\mathbf{w}\|^2$$



Logistic regression vs. SVMs

n =number of features ($x \in \mathbb{R}^{n+1}$), m =number of training examples

If n is large (relative to m): n>=m ; n=10000; m=10,...,1000

Use logistic regression, or SVM without a kernel (“linear kernel”)

If n is small, m is intermediate: n=1,...,1000 ; m=10,..., 10000

Use SVM with Gaussian kernel

If n is small, m is large: n=1,...,1000; m>=50000

Create/add more features, then use logistic regression or SVM without a kernel

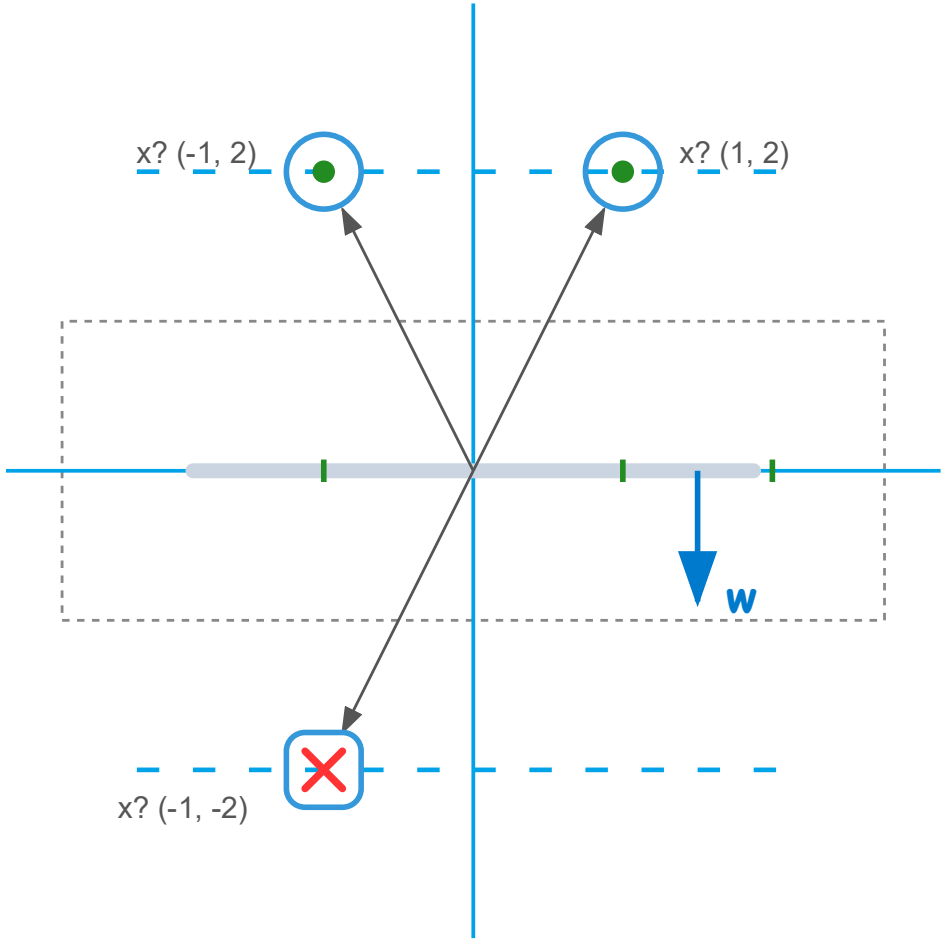
Neural network likely to work well for most of these settings, but may be slower to train.

SVM Example

- **Point 1:** $x_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad y_1 = -1$
- **Point 2:** $x_2 = \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \quad y_2 = -1$
- **Point 3:** $x_3 = \begin{pmatrix} -1 \\ -2 \end{pmatrix}, \quad y_3 = 1$

$$L(w, b, \alpha) = \frac{1}{2}\|w\|_2^2 - \sum_{i=1}^{n} \alpha_i \left(y_i (x^T_i w + b) - 1 \right)$$

$$L(w, b, \alpha) = \frac{1}{2} w^T w - \sum_{i=1}^{n} \alpha_i y_i x^T_i w - b \left(\sum_{i=1}^{n} \alpha_i y_i \right) + \sum_{i=1}^{n} \alpha_i$$



Taking Derivatives (KKT Stationarity)

Equation ① (Derivative with respect to w):

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \implies w^* = \sum_{i=1}^{n} \alpha_i y_i x_i$$

Equation ② (Derivative with respect to b):

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \implies \sum_{i=1}^{n} \alpha_i y_i = 0$$

Substituting into Equation ① (w^*) : Since $y_1 = -1$,$y_2 = -1$,and $y_3 = 1$,we get

$$w^* = \sum_{i=1}^{3} \alpha_i y_i x_i = -\alpha_1 \begin{pmatrix} 1 \\ 2 \end{pmatrix} - \alpha_2 \begin{pmatrix} -1 \\ 2 \end{pmatrix} + \alpha_3 \begin{pmatrix} -1 \\ -2 \end{pmatrix}$$

Substituting into Equation ②:

$$-\alpha_1 - \alpha_2 + \alpha_3 = 0$$





Formulating the Dual Problem

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$x_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, x_2 = \begin{pmatrix} -1 \\ 2 \end{pmatrix}, x_3 = \begin{pmatrix} -1 \\ -2 \end{pmatrix}:$$

- $x_1^T x_1 = (1)(1) + (2)(2) = 5$
- $x_1^T x_2 = (1)(-1) + (2)(2) = 3$
- $x_1^T x_3 = (1)(-1) + (2)(-2) = -5$
- $x_2^T x_2 = (-1)(-1) + (2)(2) = 5$
- $x_2^T x_3 = (-1)(-1) + (2)(-2) = -3$
- $x_3^T x_3 = (-1)(-1) + (-2)(-2) = 5$

$$\max_{\alpha} -\frac{1}{2} \left[5\alpha_1^2 + 6\alpha_1\alpha_2 + 10\alpha_1\alpha_3 + 5\alpha_2^2 + 6\alpha_2\alpha_3 + 5\alpha_3^2 \right] + \alpha_1 + \alpha_2 + \alpha_3$$

$$y_1 = -1, y_2 = -1, y_3 = 1$$





Solving the Optimization

$$\max_{\alpha} -\frac{1}{2}\left[5\alpha_1^2 + 6\alpha_1\alpha_2 + 10\alpha_1\alpha_3 + 5\alpha_2^2 + 6\alpha_2\alpha_3 + 5\alpha_3^2\right] + \alpha_1 + \alpha_2 + \alpha_3$$

$$\sum_{i=1}^{3} \alpha_i y_i = 0 \implies -\alpha_1 - \alpha_2 + \alpha_3 = 0 \implies \alpha_3 = \alpha_1 + \alpha_2$$

$$J = \max_{\alpha_1,\alpha_2} -\frac{1}{2}\left(20\alpha_1^2 + 32\alpha_1\alpha_2 + 16\alpha_2^2\right) + 2\alpha_1 + 2\alpha_2$$

1. $$\frac{\partial J}{\partial \alpha_1} = -20\alpha_1 - 16\alpha_2 + 2 = 0 \implies \alpha_1 = \frac{-16\alpha_2 + 2}{20}$$ $\alpha_1 = 0$

2. $$\frac{\partial J}{\partial \alpha_2} = -16\alpha_1 - 16\alpha_2 + 2 = 0$$ $\alpha_2 = \frac{1}{8}$

$$\alpha_3 = \alpha_1 + \alpha_2$$ $\alpha_3 = \frac{1}{8}$





Calculating Optimal Weights and Margin

$$w^* = \sum_{i=1}^{3} \alpha_i y_i x_i = \alpha_1 y_1 x_1 + \alpha_2 y_2 x_2 + \alpha_3 y_3 x_3$$

$$w^* = 0 - \frac{1}{8}\begin{pmatrix}-1\\2\end{pmatrix} + \frac{1}{8}\begin{pmatrix}-1\\-2\end{pmatrix} = \begin{pmatrix}1/8\\-2/8\end{pmatrix} + \begin{pmatrix}-1/8\\-2/8\end{pmatrix} = \begin{pmatrix}0\\-1/2\end{pmatrix}$$

$$\|w\| = \sqrt{0^2 + (-1/2)^2} = 1/2$$

$$m = \frac{2}{\|w\|} = \frac{2}{1/2} = 4$$





Calculating the Bias b^*

Let's use Support Vector x_2 (where $\alpha_2 = 1/8$ and $y_2 = -1$):

$$y_2(x_2^T w^* + b^*) = 1$$

Since $y_2 = -1$, we can divide both sides by -1:

$$x_2^T w^* + b^* = -1$$

Substitute the values for x_2 and w^*:

$$b^* = -1 - \begin{pmatrix} -1 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ -1/2 \end{pmatrix}$$

$$b^* = -1 - ((-1)(0) + (2)(-1/2))$$

$$b^* = -1 - (-1)$$

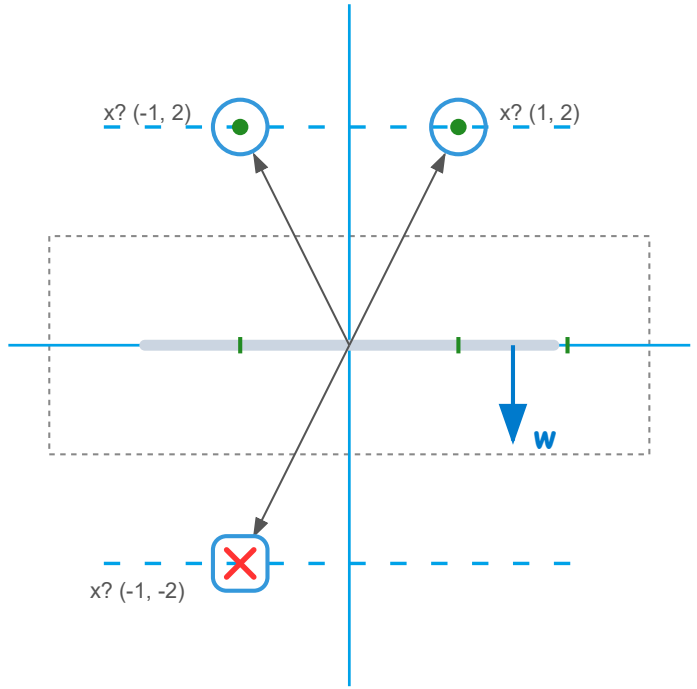
$$b^* = 0$$





Final Model Summary:

The optimal separating hyperplane is defined by $w^* = \begin{pmatrix} 0 \\ -1/2 \end{pmatrix}$ and $b^* = 0$. The decision boundary equation is simply $-0.5x^{(2)} = 0$ (or $x^{(2)} = 0$, the x-axis), and the margin width is 4.



References and further readings

- Alexandre Kowalczyk, **SVM Tutorial**, **11 may 2020,** https://www.svm-tutorial.com/

- Alexandre Kowalczyk, “Support vector machines succinctly”, syncfusion, Inc,
- Martin Hafmann, “kernels and the kernel trick”, 11 may 2020, https://cogsys.uni-bamberg.de/teaching/ss06/hs_svm/slides/SVM_and_Kernels.pdf
- Andrew NG., Machine Learning Course, Coursera, slide: Support Vector Machines
- Tengyu Ma and Andrew Ng, CS229 Lecture Notes, 2020, http://cs229.stanford.edu/notes2020fall/notes2020fall/cs229-notes3.pdf





- Ryan Tibshirani, Lecture 12: KKT Conditions, convex optimization course, spring 2015, https://www.stat.cmu.edu/~ryantibs/convexopt-S15/scribes/12-kkt-scribed.pdf
- Bishop , pattern recognition and machine learning



